```
# ============================================================
# SECTION 1: RESEARCH ENVIRONMENT VERIFICATION
# ============================================================
import torch

print("PyTorch Version :", torch.__version__)
print("CUDA Available  :", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU Device      :", torch.cuda.get_device_name(0))
else:
    print("WARNING: No GPU detected. BERT fine-tuning will be slow.")
    print("Go to Runtime > Change runtime type > select GPU (T4/L4/A100).")
```

```
PyTorch Version : 2.11.0+cu128
CUDA Available  : True
GPU Device      : NVIDIA L4
```

```
# ============================================================
# SECTION 2: INSTALL REQUIRED PACKAGES
# ============================================================
!pip install -q scikit-learn xgboost joblib seaborn
```

```
# ============================================================
# SECTION 3: IMPORT CORE LIBRARIES
# ============================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

print("Core libraries loaded successfully.")
print("Random seed set to:", RANDOM_SEED)
```

```
Core libraries loaded successfully.
Random seed set to: 42
```

```
# ============================================================
# SECTION 4: UPLOAD DATASET
# ============================================================
from google.colab import files

print("Please select malicious_phish.csv from your computer...")
uploaded = files.upload()

DATA_PATH = list(uploaded.keys())[0]
print("File uploaded:", DATA_PATH)
```

```
Please select malicious_phish.csv from your computer...
Choose Files malicious_phish.csv
malicious_phish.csv(text/csv) - 45664459 bytes, last modified: 6/21/2026 - 100% done
Saving malicious_phish.csv to malicious_phish.csv
File uploaded: malicious_phish.csv
```

```
# ============================================================
# SECTION 5: LOAD DATASET
# ============================================================
df_raw = pd.read_csv(DATA_PATH)

print("Raw dataset shape:", df_raw.shape)
df_raw.head()
```

```
Raw dataset shape: (651191, 2)
```

	url	type
0	br-icloud.com.br	phishing
1	mp3raid.com/music/krizz_kaliko.html	benign
2	bopsecrets.org/rexroth/cr/1.htm	benign
3	http://www.garage-pirenne.be/index.php?option=...	defacement
4	http://adventure-nicaragua.net/index.php?optio...	defacement

```
# ============================================================
# SECTION 6: DATASET STRUCTURE CHECK
# ============================================================
print(df_raw.info())
print()
print("Columns:", list(df_raw.columns))
```

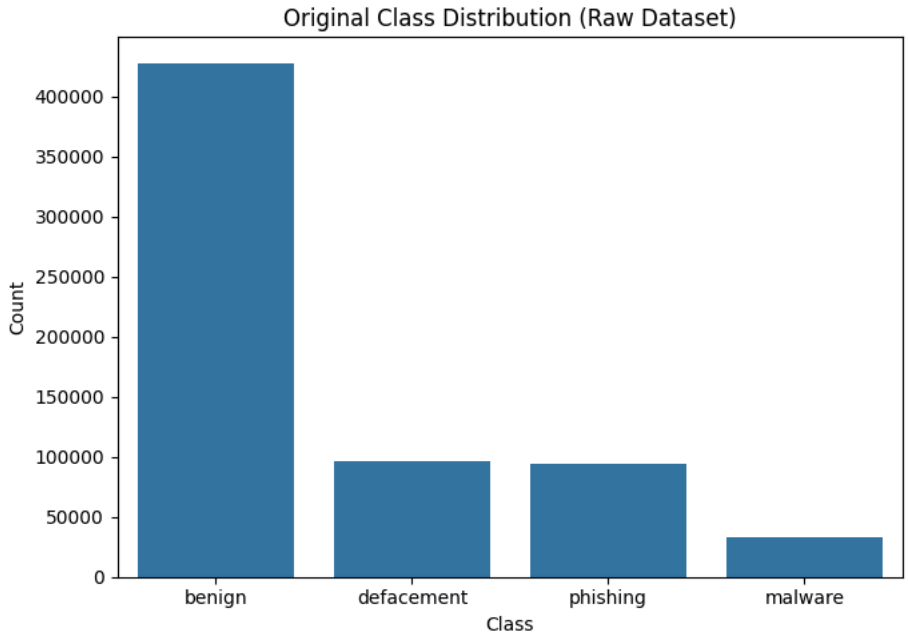
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 651191 entries, 0 to 651190
Data columns (total 2 columns):
 #   Column  Non-Null Count   Dtype 
---  ------  --------------   ----- 
 0   url     651191 non-null  object
 1   type    651191 non-null  object
dtypes: object(2)
memory usage: 9.9+ MB
None

Columns: ['url', 'type']
```

```
# ============================================================
# SECTION 7: MISSING VALUES CHECK
# ============================================================
missing_counts = df_raw.isnull().sum()
print("Missing values per column:")
print(missing_counts)

total_missing = missing_counts.sum()
print(f"\nTotal missing values: {total_missing}")
```

```
Missing values per column:
url     0
type    0
dtype: int64

Total missing values: 0
```

```
# ============================================================
# SECTION 8: ORIGINAL CLASS DISTRIBUTION (BEFORE CLEANING)
# ============================================================
print("Original class distribution:")
print(df_raw["type"].value_counts())

plt.figure(figsize=(7,5))
sns.countplot(x=df_raw["type"], order=df_raw["type"].value_counts().index)
plt.title("Original Class Distribution (Raw Dataset)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("Figure0_Raw_Class_Distribution.png", dpi=300)
plt.show()
```

```
Original class distribution:
type
benign        428103
defacement     96457
phishing       94111
malware        32520
Name: count, dtype: int64
```

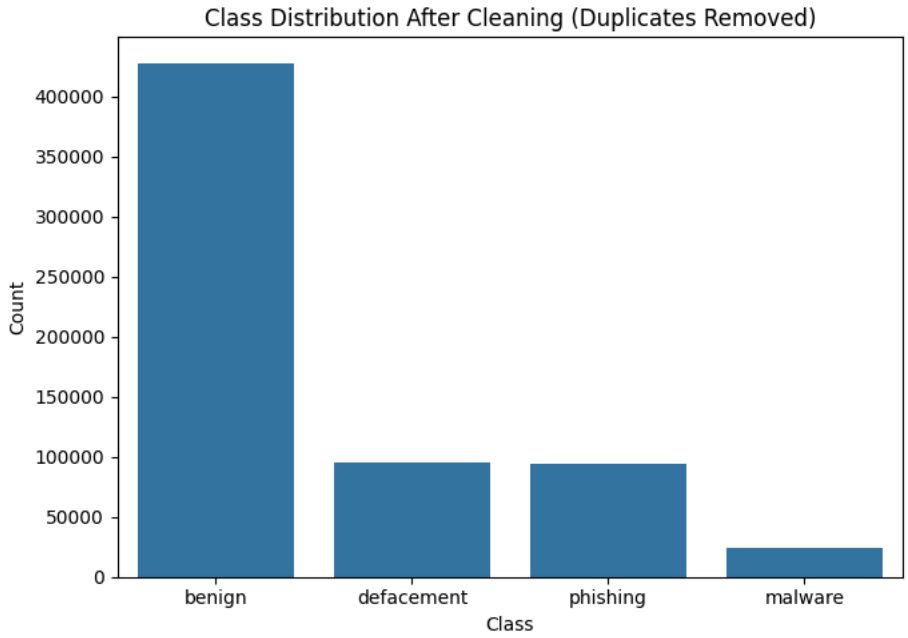



```
# ============================================================
# SECTION 9: DATA CLEANING
# Remove missing values and duplicate URLs
# ============================================================
df_clean = df_raw.dropna(subset=["url", "type"]).copy()
print("Shape after dropping missing values:", df_clean.shape)

before_dedup = len(df_clean)
df_clean = df_clean.drop_duplicates(subset=["url"]).reset_index(drop=True)
after_dedup = len(df_clean)

print(f"Shape after removing duplicate URLs: {df_clean.shape}")
print(f"Duplicates removed: {before_dedup - after_dedup}")
```

```
Shape after dropping missing values: (651191, 2)
Shape after removing duplicate URLs: (641119, 2)
Duplicates removed: 10072
```

```
# ============================================================
# SECTION 10: CLEANED CLASS DISTRIBUTION
# ============================================================
print("Cleaned class distribution:")
print(df_clean["type"].value_counts())

plt.figure(figsize=(7,5))
sns.countplot(x=df_clean["type"], order=df_clean["type"].value_counts().index)
plt.title("Class Distribution After Cleaning (Duplicates Removed)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("Figure0b_Cleaned_Class_Distribution.png", dpi=300)
plt.show()
```

```
Cleaned class distribution:
type
benign        428080
defacement     95308
phishing       94086
malware        23645
Name: count, dtype: int64
```




```
# ============================================================
# SECTION 11: SAVE CLEANED DATASET
# ============================================================
df_clean.to_csv("Cleaned_Malicious_URL_Dataset.csv", index=False)
print("Cleaned dataset saved as: Cleaned_Malicious_URL_Dataset.csv")
print("Final cleaned shape:", df_clean.shape)
```

```
Cleaned dataset saved as: Cleaned_Malicious_URL_Dataset.csv
Final cleaned shape: (641119, 2)
```

```
# ============================================================
# SECTION 12: BINARY LABEL CONVERSION
# Benign -> Safe (0)
# Phishing, Malware, Defacement -> Unsafe (1)
# ============================================================
safe_df = df_clean[df_clean["type"] == "benign"].copy()

unsafe_df = df_clean[
    df_clean["type"].isin(["phishing", "malware", "defacement"])
].copy()

print("Safe (Benign) samples   :", len(safe_df))
print("Unsafe (Phish/Mal/Def) samples:", len(unsafe_df))
```

```
Safe (Benign) samples   : 428080
Unsafe (Phish/Mal/Def) samples: 213039
```

```
# ============================================================
# SECTION 13: BALANCED 50K SAMPLING
# 25,000 Safe + 25,000 Unsafe, random_state=42
# ============================================================
RANDOM_SEED = 42

safe_sample = safe_df.sample(n=25000, random_state=RANDOM_SEED)
unsafe_sample = unsafe_df.sample(n=25000, random_state=RANDOM_SEED)

safe_sample["label"] = 0
unsafe_sample["label"] = 1

binary_df = pd.concat([safe_sample, unsafe_sample])
binary_df = binary_df.sample(frac=1, random_state=RANDOM_SEED).reset_index(drop=True)
```

```
print("Balanced dataset shape:", binary_df.shape)
binary_df.head()
```

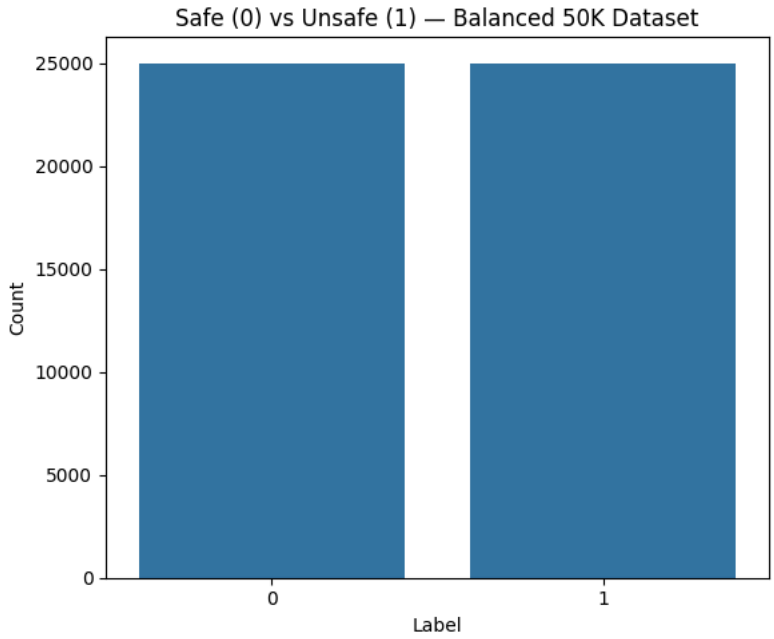
```
Balanced dataset shape: (50000, 3)
```

	url	type	label
0	http://37.49.225.137/HARRY%20B_mcvGZe134.bin	malware	1
1	at1ce.org/themenreihe.p?c=People%20of%20Indian...	benign	0
2	pichinchaas.irtvmarket.com/internexo/jsp/login...	benign	0
3	answers.com/topic/expo-67	benign	0
4	http://www.techlimits.com/contactos	defacement	1

Next steps: Generate code with `binary_df` New interactive sheet

```
# ============================================================
# SECTION 14: VERIFY BALANCE AND SAVE
# ============================================================
print("Label distribution:")
print(binary_df["label"].value_counts())

plt.figure(figsize=(6,5))
sns.countplot(x=binary_df["label"])
plt.title("Safe (0) vs Unsafe (1) — Balanced 50K Dataset")
plt.xlabel("Label")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig("Figure0c_Balanced_Distribution.png", dpi=300)
plt.show()

binary_df.to_csv("Balanced_50K_URL_Dataset.csv", index=False)
print("Saved: Balanced_50K_URL_Dataset.csv")
```

```
Label distribution:
label
1    25000
0    25000
Name: count, dtype: int64
```




```
Saved: Balanced_50K_URL_Dataset.csv
```

```
# ============================================================
# SECTION 15: TRAIN/TEST SPLIT
# 80% Train / 20% Test, Stratified, random_state=42
# ============================================================
from sklearn.model_selection import train_test_split

train_texts, test_texts, train_labels, test_labels = train_test_split(
    binary_df["url"],
```

```
    binary_df["label"],
    test_size=0.20,
    random_state=RANDOM_SEED,
    stratify=binary_df["label"]
)

print("Train samples:", len(train_texts))
print("Test samples :", len(test_texts))
```

```
Train samples: 40000
Test samples : 10000
```

```
# ============================================================
# SECTION 16: SAVE TRAIN/TEST SPLIT
# ============================================================
import joblib

train_df_out = pd.DataFrame({"url": train_texts, "label": train_labels})
test_df_out  = pd.DataFrame({"url": test_texts,  "label": test_labels})

train_df_out.to_csv("Train_40K.csv", index=False)
test_df_out.to_csv("Test_10K.csv", index=False)

joblib.dump(
    (train_texts, test_texts, train_labels, test_labels),
    "binary_train_test_split.pkl"
)

print("Saved: Train_40K.csv, Test_10K.csv, binary_train_test_split.pkl")
print("Train label distribution:")
print(train_labels.value_counts())
print("Test label distribution:")
print(test_labels.value_counts())
```

```
Saved: Train_40K.csv, Test_10K.csv, binary_train_test_split.pkl
Train label distribution:
label
1    20000
0    20000
Name: count, dtype: int64
Test label distribution:
label
0    5000
1    5000
Name: count, dtype: int64
```

```
# ============================================================
# SECTION 17: INSTALL TRANSFORMERS STACK
# ============================================================
!pip install -q transformers datasets accelerate evaluate
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 9.0 MB/s eta 0:00:00
```

```
# ============================================================
# SECTION 18: IMPORTS FOR BERT FINE-TUNING
# ============================================================
import numpy as np
import pandas as pd
import torch
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding
)
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support
)

RANDOM_SEED = 42
torch.manual_seed(RANDOM_SEED)

print("Transformers stack imported. Seed set to", RANDOM_SEED)
```

```
Transformers stack imported. Seed set to 42
```

```
# ============================================================
# SECTION 19: BUILD HUGGING FACE DATASETS
# ============================================================
train_df = pd.DataFrame({
    "text": train_texts.tolist(),
    "label": train_labels.tolist()
})
test_df = pd.DataFrame({
    "text": test_texts.tolist(),
    "label": test_labels.tolist()
})

train_dataset = Dataset.from_pandas(train_df)
test_dataset = Dataset.from_pandas(test_df)

print(train_dataset)
print(test_dataset)
```

```
Dataset({
    features: ['text', 'label'],
    num_rows: 40000
})
Dataset({
    features: ['text', 'label'],
    num_rows: 10000
})
```

```
# ============================================================
# SECTION 20: LOAD BERT TOKENIZER
# ============================================================
MODEL_NAME = "bert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
print("Tokenizer loaded:", MODEL_NAME)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
config.json: 100%  570/570 [00:00<00:00, 77.4kB/s]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 6.55kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 3.14MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 2.27MB/s]
Tokenizer loaded: bert-base-uncased
```

```
# ============================================================
# SECTION 21: TOKENIZE DATASETS (max_length=128)
# ============================================================
def tokenize_function(examples):
    return tokenizer(
        examples["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

tokenized_train = train_dataset.map(tokenize_function, batched=True)
tokenized_test = test_dataset.map(tokenize_function, batched=True)

tokenized_train = tokenized_train.remove_columns(["text"])
tokenized_test = tokenized_test.remove_columns(["text"])

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

print("Tokenization complete.")
print(tokenized_train)
```

```
Map: 100%                                                    40000/40000 [00:03<00:00, 10883.27 examples/s]
Map: 100%                                                    10000/10000 [00:00<00:00, 11804.47 examples/s]
Tokenization complete.
Dataset({
    features: ['label', 'input_ids', 'token_type_ids', 'attention_mask'],
    num_rows: 40000
})
```

```
# ============================================================
# SECTION 22: LOAD BERT MODEL FOR SEQUENCE CLASSIFICATION
# ============================================================
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=2
)

print("Model loaded:", MODEL_NAME)
```

```
model.safetensors: 100%                                      440M/440M [00:05<00:00, 224MB/s]
Loading weights: 100%                                        199/199 [00:00<00:00, 3347.32it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
classifier.weight                           | MISSING    |
classifier.bias                             | MISSING    |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
Model loaded: bert-base-uncased
```

```
# ============================================================
# SECTION 23: DEFINE EVALUATION METRICS
# ============================================================
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="weighted"
    )
    accuracy = accuracy_score(labels, predictions)

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

```
# ============================================================
# SECTION 24: TRAINING ARGUMENTS
# Seed=42, Epochs=2, LR=2e-5, Batch=16, Weight Decay=0.01
# ============================================================
training_args = TrainingArguments(
    output_dir="./bert_binary_results",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    seed=RANDOM_SEED,
    fp16=True,
```

```
    logging_strategy="epoch"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_test,
    data_collator=data_collator,
    compute_metrics=compute_metrics
)

print("Trainer configured.")
```

```
Trainer configured.
```

```
# ============================================================
# SECTION 25: TRAIN BERT
# ============================================================
train_result = trainer.train()
print(train_result)
```

[5000/5000 06:19, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.137394	0.089646	0.975600	0.975683	0.975600	0.975599
2	0.056831	0.094273	0.977700	0.977750	0.977700	0.977699

Writing model shards: 100% 1/1 [00:00<00:00, 1.85it/s]

Writing model shards: 100% 1/1 [00:00<00:00, 1.82it/s]

```
[transformers] There were missing keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.weight', 'bert.embeddings.Lay
[transformers] There were unexpected keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.beta', 'bert.embeddings.La
TrainOutput(global_step=5000, training_loss=0.09711247863769532, metrics={'train_runtime': 381.109, 'train_samples_per_second':
```

```
# ============================================================
# SECTION 26: EVALUATE ON TEST SET
# ============================================================
eval_results = trainer.evaluate()
print(eval_results)
```

[625/625 00:12]

Validation Loss	Epoch	Accuracy	Precision	Recall	F1
0.094273	2	0.977700	0.977750	0.977700	0.977699

```
3.09427312761545181, 'eval_accuracy': 0.9777, 'eval_precision': 0.9777497050793165, 'eval_recall': 0.9777, 'eval_f1
```

```
# ============================================================
# SECTION 27: GENERATE PREDICTIONS ON TEST SET
# ============================================================
predictions_output = trainer.predict(tokenized_test)

y_true = predictions_output.label_ids
y_pred = np.argmax(predictions_output.predictions, axis=1)

# Softmax probabilities for ROC/PR curves (probability of class 1 = Unsafe)
from scipy.special import softmax
y_probs = softmax(predictions_output.predictions, axis=1)[:, 1]

print("Predictions generated.")
print("Shapes -> y_true:", y_true.shape, "y_pred:", y_pred.shape, "y_probs:", y_probs.shape)
```

```
Predictions generated.
Shapes -> y_true: (10000,) y_pred: (10000,) y_probs: (10000,)
```

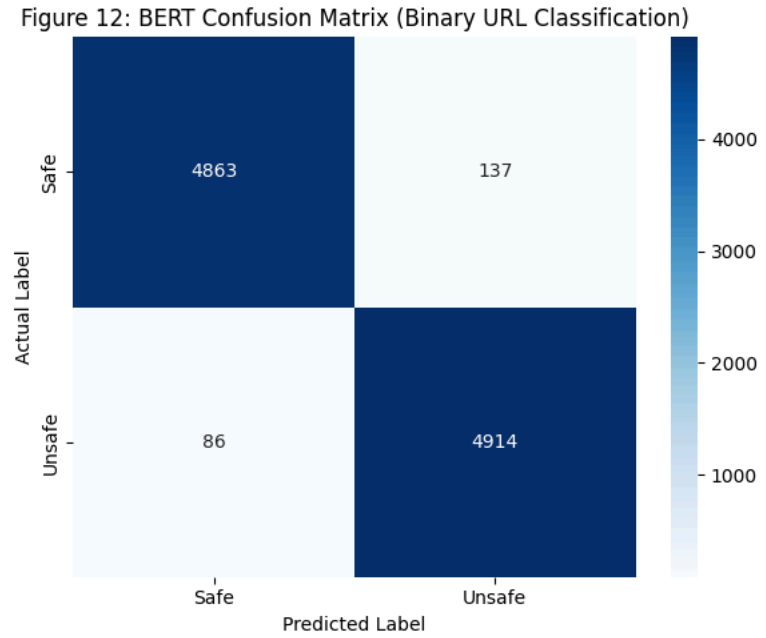
```
# ============================================================
# SECTION 28: CLASSIFICATION REPORT
# ============================================================
from sklearn.metrics import classification_report

report = classification_report(
    y_true, y_pred, target_names=["Safe", "Unsafe"]
```

```
)
print(report)

with open("BERT_Classification_Report.txt", "w") as f:
    f.write(report)
```

```
              precision    recall  f1-score   support

        Safe       0.98      0.97      0.98      5000
      Unsafe       0.97      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000
```

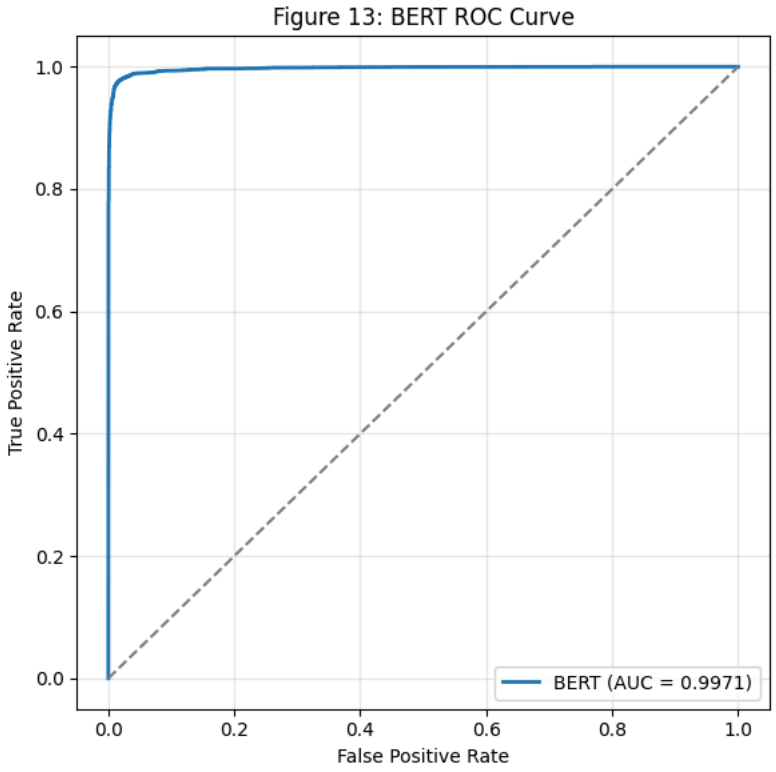
```
# ============================================================
# SECTION 29: CONFUSION MATRIX
# ============================================================
from sklearn.metrics import confusion_matrix

cm_bert = confusion_matrix(y_true, y_pred)
print(cm_bert)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm_bert, annot=True, fmt="d", cmap="Blues",
    xticklabels=["Safe","Unsafe"], yticklabels=["Safe","Unsafe"]
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Figure 12: BERT Confusion Matrix (Binary URL Classification)")
plt.tight_layout()
plt.savefig("Figure12_BERT_Confusion_Matrix.png", dpi=300)
plt.show()
```

```
[[4863  137]
 [  86 4914]]
```



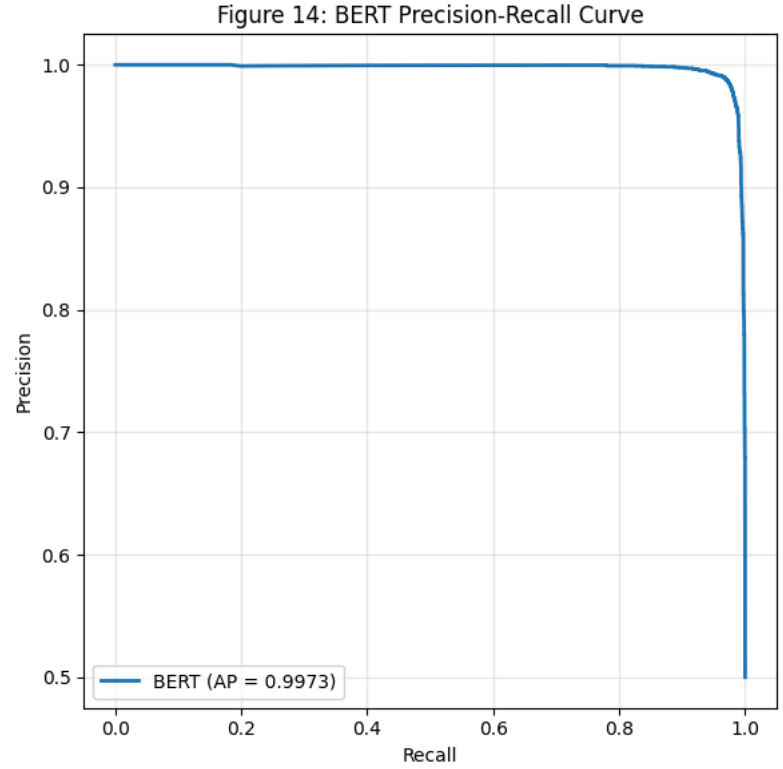

```
# ============================================================
# SECTION 30: ROC CURVE
# ============================================================
from sklearn.metrics import roc_curve, auc

fpr, tpr, _ = roc_curve(y_true, y_probs)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, linewidth=2, label=f"BERT (AUC = {roc_auc:.4f})")
plt.plot([0,1],[0,1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
plt.title("Figure 13: BERT ROC Curve")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Figure13_BERT_ROC_Curve.png", dpi=300)
plt.show()

print("BERT ROC-AUC:", roc_auc)
```



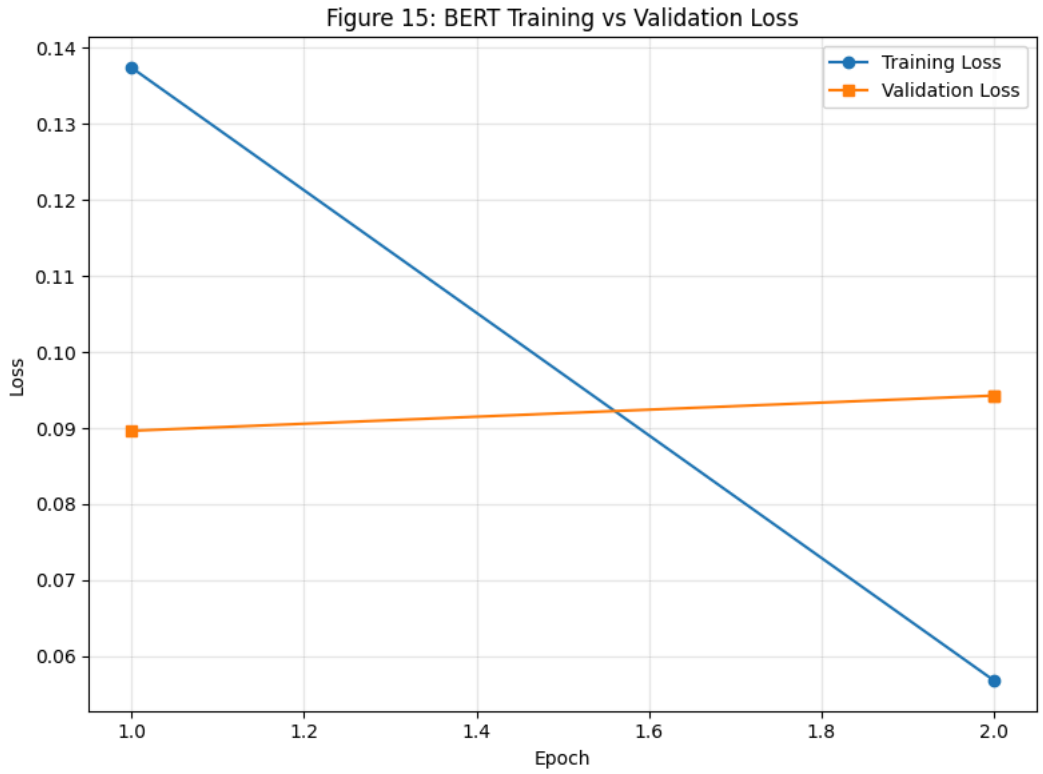

```
BERT ROC-AUC: 0.99712568
```

```
# ============================================================
# SECTION 31: PRECISION-RECALL CURVE
# ============================================================
from sklearn.metrics import precision_recall_curve, average_precision_score

prec_curve, rec_curve, _ = precision_recall_curve(y_true, y_probs)
avg_prec = average_precision_score(y_true, y_probs)

plt.figure(figsize=(6,6))
plt.plot(rec_curve, prec_curve, linewidth=2, label=f"BERT (AP = {avg_prec:.4f})")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Figure 14: BERT Precision-Recall Curve")
plt.legend(loc="lower left")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Figure14_BERT_PR_Curve.png", dpi=300)
plt.show()

print("BERT Average Precision:", avg_prec)
```




```
BERT Average Precision: 0.9972812645253164
```

```
# ============================================================
# SECTION 32: LEARNING CURVE (TRAINING/VALIDATION LOSS)
# ============================================================
log_history = trainer.state.log_history

train_epochs, train_losses = [], []
eval_epochs, eval_losses = [], []

for entry in log_history:
    if "loss" in entry and "epoch" in entry:
        train_epochs.append(entry["epoch"])
        train_losses.append(entry["loss"])
    if "eval_loss" in entry and "epoch" in entry:
        eval_epochs.append(entry["epoch"])
        eval_losses.append(entry["eval_loss"])

plt.figure(figsize=(8,6))
plt.plot(train_epochs, train_losses, marker="o", label="Training Loss")
plt.plot(eval_epochs, eval_losses, marker="s", label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Figure 15: BERT Training vs Validation Loss")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Figure15_BERT_Learning_Curve.png", dpi=300)
plt.show()

history_df = pd.DataFrame(log_history)
history_df.to_csv("BERT_Training_History.csv", index=False)
print("Training history saved: BERT_Training_History.csv")
```




```
Training history saved: BERT_Training_History.csv
```

```
# ============================================================
# SECTION 33: SAVE MODEL, TOKENIZER, AND EVALUATION CSV
# ============================================================
eval_summary = pd.DataFrame([{
    "Model": "BERT (bert-base-uncased)",
    "Accuracy": eval_results["eval_accuracy"],
    "Precision": eval_results["eval_precision"],
    "Recall": eval_results["eval_recall"],
    "F1": eval_results["eval_f1"],
    "ROC_AUC": roc_auc,
    "Average_Precision": avg_prec
}])
eval_summary.to_csv("BERT_Evaluation_Summary.csv", index=False)

BEST_MODEL_DIR = "bert_binary_best_model"
model.save_pretrained(BEST_MODEL_DIR)
tokenizer.save_pretrained(BEST_MODEL_DIR)

print("Saved best model + tokenizer to:", BEST_MODEL_DIR)
print(eval_summary)
```

```
Writing model shards: 100%                                                  1/1 [00:00<00:00,  1.93it/s]
Saved best model + tokenizer to: bert_binary_best_model
                    Model  Accuracy  Precision  Recall        F1   ROC_AUC  \
0  BERT (bert-base-uncased)    0.9777    0.97775  0.9777  0.977699  0.997126

   Average_Precision
0           0.997281
```

```
# ============================================================
# SECTION 34: RELOAD BEST BERT MODEL (EMBEDDING MODE)
# ============================================================
from transformers import AutoTokenizer, AutoModel
import torch

BEST_MODEL_DIR = "bert_binary_best_model"

embed_tokenizer = AutoTokenizer.from_pretrained(BEST_MODEL_DIR)
embed_model = AutoModel.from_pretrained(BEST_MODEL_DIR)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
embed_model.to(device)
embed_model.eval()
```

```
embed_model.eval()

print("Model reloaded for embedding extraction on:", device)
```

```
Loading weights: 100%                                                    199/199 [00:00<00:00, 4188.58it/s]
[transformers] BertModel LOAD REPORT from: bert_binary_best_model
Key               | Status     |  |
------------------+------------+--+-
classifier.bias   | UNEXPECTED |  |
classifier.weight | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Model reloaded for embedding extraction on: cuda
```

```
# ============================================================
# SECTION 35: CLS EMBEDDING EXTRACTION FUNCTION
# ============================================================
from tqdm import tqdm
import numpy as np

def generate_embeddings(texts, batch_size=32, max_length=128):
    embeddings = []
    with torch.no_grad():
        for i in tqdm(range(0, len(texts), batch_size)):
            batch = [str(t) for t in texts[i:i+batch_size]]
            inputs = embed_tokenizer(
                batch,
                padding="max_length",
                truncation=True,
                max_length=max_length,
                return_tensors="pt"
            )
            inputs = {k: v.to(device) for k, v in inputs.items()}
            outputs = embed_model(**inputs)
            cls_embeddings = outputs.last_hidden_state[:, 0, :].cpu().numpy()
            embeddings.append(cls_embeddings)
    return np.vstack(embeddings)

print("Embedding function ready (batched for speed).")
```

```
Embedding function ready (batched for speed).
```

```
# ============================================================
# SECTION 36: EXTRACT TRAIN EMBEDDINGS
# ============================================================
X_train_bert = generate_embeddings(train_texts.tolist())
print("X_train_bert shape:", X_train_bert.shape)
```

```
100%|██████████| 1250/1250 [02:10<00:00,  9.58it/s]X_train_bert shape: (40000, 768)
```

```
# ============================================================
# SECTION 37: EXTRACT TEST EMBEDDINGS
# ============================================================
X_test_bert = generate_embeddings(test_texts.tolist())
print("X_test_bert shape:", X_test_bert.shape)
```

```
100%|██████████| 313/313 [00:32<00:00,  9.63it/s]X_test_bert shape: (10000, 768)
```

```
# ============================================================
# SECTION 38: SAVE EMBEDDINGS AND LABELS
# ============================================================
np.save("X_train_bert_binary.npy", X_train_bert)
np.save("X_test_bert_binary.npy", X_test_bert)
np.save("train_labels_binary.npy", np.array(train_labels))
np.save("test_labels_binary.npy", np.array(test_labels))

print("Saved:")
print(" - X_train_bert_binary.npy", X_train_bert.shape)
print(" - X_test_bert_binary.npy ", X_test_bert.shape)
print(" - train_labels_binary.npy", len(train_labels))
print(" - test_labels_binary.npy ", len(test_labels))
```

```
Saved:
 - X_train_bert_binary.npy (40000, 768)
 - X_test_bert_binary.npy  (10000, 768)
 - train_labels_binary.npy 40000
 - test_labels_binary.npy  10000
```

```
# ============================================================
# SECTION 39: IMPORTS FOR CLASSICAL ML EXPERIMENTS
# ============================================================
import numpy as np
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix,
    roc_curve, auc, precision_recall_curve, average_precision_score
)

RANDOM_SEED = 42

# Reload labels as plain arrays for sklearn compatibility
train_labels_arr = np.load("train_labels_binary.npy")
test_labels_arr = np.load("test_labels_binary.npy")

print("Setup complete. Train/test label arrays loaded.")
```

```
Setup complete. Train/test label arrays loaded.
```

```
# ============================================================
# SECTION 40: BERT EMBEDDINGS + SVM
# ============================================================
svm_model = SVC(kernel="rbf", C=1.0, gamma="scale", probability=True, random_state=RANDOM_SEED)
svm_model.fit(X_train_bert, train_labels_arr)

svm_preds = svm_model.predict(X_test_bert)
svm_probs = svm_model.predict_proba(X_test_bert)[:, 1]

svm_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, svm_preds),
    "Precision": precision_score(test_labels_arr, svm_preds),
    "Recall": recall_score(test_labels_arr, svm_preds),
    "F1": f1_score(test_labels_arr, svm_preds)
}
print("BERT + SVM:", svm_metrics)
print(classification_report(test_labels_arr, svm_preds, target_names=["Safe","Unsafe"]))

cm_svm = confusion_matrix(test_labels_arr, svm_preds)
print(cm_svm)

np.save("bert_svm_predictions.npy", svm_preds)
np.save("bert_svm_probs.npy", svm_probs)
```

```
BERT + SVM: {'Accuracy': 0.9793, 'Precision': 0.9792041591681664, 'Recall': 0.9794, 'F1': 0.9793020697930207}
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000

[[4896  104]
 [ 103 4897]]
```

```
# ============================================================
# SECTION 41: BERT EMBEDDINGS + XGBOOST
# ============================================================
xgb_bert = XGBClassifier(
    n_estimators=200, max_depth=6, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8,
    random_state=RANDOM_SEED, eval_metric="logloss"
)
xgb_bert.fit(X_train_bert, train_labels_arr)

xgb_bert_preds = xgb_bert.predict(X_test_bert)
```

```
xgb_bert_probs = xgb_bert.predict_proba(X_test_bert)[:, 1]

xgb_bert_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, xgb_bert_preds),
    "Precision": precision_score(test_labels_arr, xgb_bert_preds),
    "Recall": recall_score(test_labels_arr, xgb_bert_preds),
    "F1": f1_score(test_labels_arr, xgb_bert_preds)
}
print("BERT + XGBoost:", xgb_bert_metrics)
print(classification_report(test_labels_arr, xgb_bert_preds, target_names=["Safe","Unsafe"]))

cm_xgb_bert = confusion_matrix(test_labels_arr, xgb_bert_preds)
print(cm_xgb_bert)

np.save("bert_xgb_predictions.npy", xgb_bert_preds)
np.save("bert_xgb_probs.npy", xgb_bert_probs)
```

```
BERT + XGBoost: {'Accuracy': 0.9797, 'Precision': 0.9801801801801802, 'Recall': 0.9792, 'F1': 0.9796898449224613}
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000

[[4901   99]
 [ 104 4896]]
```

```
# ============================================================
# SECTION 42: BERT EMBEDDINGS + RANDOM FOREST
# ============================================================
rf_bert = RandomForestClassifier(n_estimators=200, random_state=RANDOM_SEED, n_jobs=-1)
rf_bert.fit(X_train_bert, train_labels_arr)

rf_bert_preds = rf_bert.predict(X_test_bert)
rf_bert_probs = rf_bert.predict_proba(X_test_bert)[:, 1]

rf_bert_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, rf_bert_preds),
    "Precision": precision_score(test_labels_arr, rf_bert_preds),
    "Recall": recall_score(test_labels_arr, rf_bert_preds),
    "F1": f1_score(test_labels_arr, rf_bert_preds)
}
print("BERT + Random Forest:", rf_bert_metrics)
print(classification_report(test_labels_arr, rf_bert_preds, target_names=["Safe","Unsafe"]))

cm_rf_bert = confusion_matrix(test_labels_arr, rf_bert_preds)
print(cm_rf_bert)

np.save("bert_rf_predictions.npy", rf_bert_preds)
np.save("bert_rf_probs.npy", rf_bert_probs)
```

```
BERT + Random Forest: {'Accuracy': 0.9789, 'Precision': 0.9788042391521695, 'Recall': 0.979, 'F1': 0.9789021097890211}
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000

[[4894  106]
 [ 105 4895]]
```

```
# ============================================================
# SECTION 43: FIGURE - CONFUSION MATRICES (SVM, XGB, RF)
# ============================================================
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 3, figsize=(18,5))

for ax, cm, title in zip(
    axes,
    [cm_svm, cm_xgb_bert, cm_rf_bert],
```

```
    ["BERT + SVM", "BERT + XGBoost", "BERT + Random Forest"]
):
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=["Safe","Unsafe"], yticklabels=["Safe","Unsafe"], ax=ax)
    ax.set_title(title)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.tight_layout()
plt.savefig("Figure16_BERT_Classifiers_Confusion_Matrices.png", dpi=300)
plt.show()

# ============================================================
# SECTION 44: FIGURE - ROC CURVE COMPARISON
# ============================================================
plt.figure(figsize=(7,7))

for probs, label in zip(
    [svm_probs, xgb_bert_probs, rf_bert_probs],
    ["BERT+SVM", "BERT+XGBoost", "BERT+RF"]
):
    fpr, tpr, _ = roc_curve(test_labels_arr, probs)
    roc_auc_val = auc(fpr, tpr)
    plt.plot(fpr, tpr, linewidth=2, label=f"{label} (AUC={roc_auc_val:.4f})")

plt.plot([0,1],[0,1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Figure 17: ROC Curve Comparison — BERT Embeddings + Classical Classifiers")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Figure17_BERT_Classifiers_ROC_Comparison.png", dpi=300)
plt.show()
```

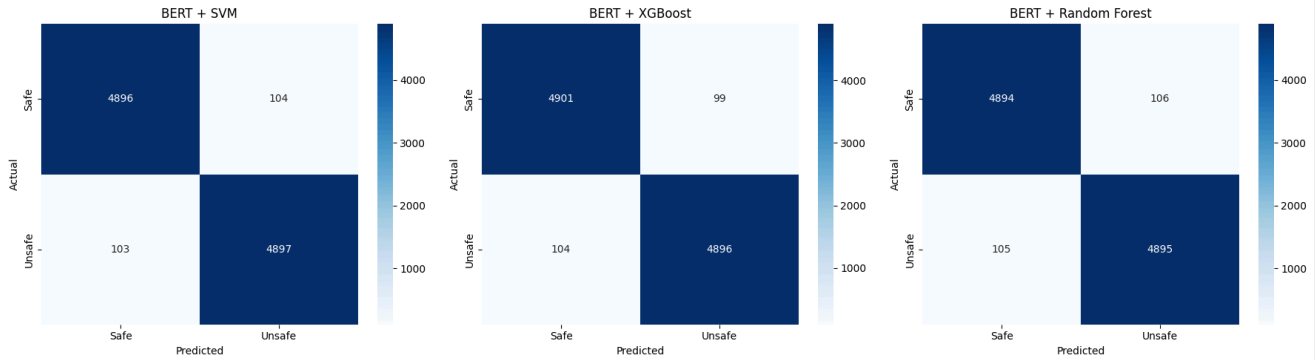
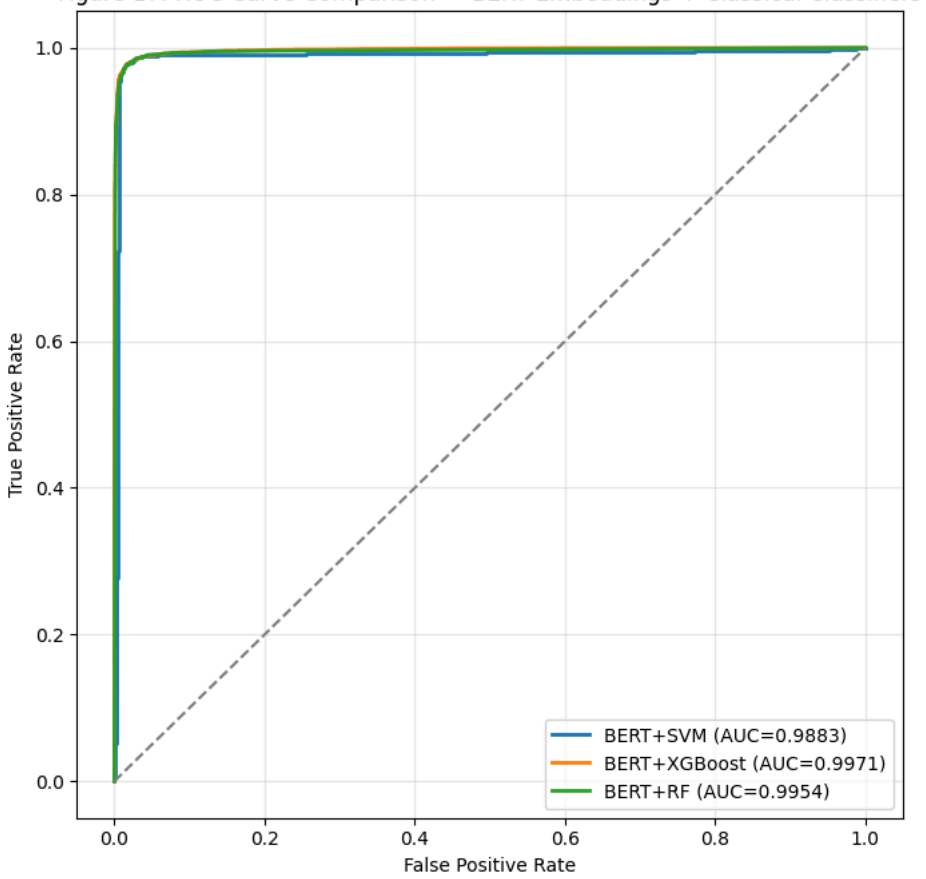


Figure 17: ROC Curve Comparison — BERT Embeddings + Classical Classifiers




```
# ============================================================
# SECTION 45: HANDCRAFTED URL FEATURE EXTRACTION FUNCTIONS
# ============================================================
from urllib.parse import urlparse
import math
from collections import Counter

def calculate_entropy(text):
    counter = Counter(text)
    length = len(text)
    entropy = 0
    for count in counter.values():
        probability = count / length
        entropy -= probability * math.log2(probability)
    return entropy

def extract_url_features(url):
    url = str(url)
    parsed = urlparse(url)
    domain = parsed.netloc
    path = parsed.path

    features = [
```

```
        len(url),                               # 1. URL length
        len(domain),                             # 2. Domain length
        len(path),                               # 3. Path length
        url.count('.'),                          # 4. Dot count
        url.count('-'),                          # 5. Hyphen count
        url.count('_'),                          # 6. Underscore count
        url.count('/'),                          # 7. Slash count
        url.count('?'),                          # 8. Question mark count
        url.count('='),                          # 9. Equals count
        sum(c.isdigit() for c in url),           # 10. Digit count
        sum(c.isalpha() for c in url),           # 11. Alphabetic char count
        url.count('@'),                          # 12. At-symbol count
        url.count('%'),                          # 13. Percent count
        url.count('&'),                          # 14. Ampersand count
        calculate_entropy(url),                  # 15. Shannon entropy
        int("https" in url),                     # 16. HTTPS presence
        int("login" in url.lower()),             # 17. "login" keyword
        int("verify" in url.lower()),            # 18. "verify" keyword
        int("secure" in url.lower()),            # 19. "secure" keyword
        int("account" in url.lower())            # 20. "account" keyword
    ]
    return features

print("Feature extraction function defined (20 handcrafted features).")
```

```
Feature extraction function defined (20 handcrafted features).
```

```
# ============================================================
# SECTION 46: EXTRACT HANDCRAFTED FEATURES
# ============================================================
X_train_url = np.array([extract_url_features(url) for url in train_texts])
X_test_url = np.array([extract_url_features(url) for url in test_texts])

print("X_train_url shape:", X_train_url.shape)
print("X_test_url shape :", X_test_url.shape)
```

```
X_train_url shape: (40000, 20)
X_test_url shape : (10000, 20)
```

```
# ============================================================
# SECTION 47: SAVE HANDCRAFTED FEATURES
# ============================================================
np.save("X_train_url_features.npy", X_train_url)
np.save("X_test_url_features.npy", X_test_url)

feature_names = [
    "url_length", "domain_length", "path_length", "dot_count", "hyphen_count",
    "underscore_count", "slash_count", "question_count", "equals_count",
    "digit_count", "alpha_count", "at_count", "percent_count", "ampersand_count",
    "entropy", "https_flag", "login_keyword", "verify_keyword",
    "secure_keyword", "account_keyword"
]

import pandas as pd
train_url_df = pd.DataFrame(X_train_url, columns=feature_names)
print(train_url_df.describe())
```

```
         url_length  domain_length   path_length     dot_count  hyphen_count  \
count  40000.000000   40000.000000  40000.000000  40000.000000  40000.000000
mean      60.978475       6.817300     37.647225      2.330925      1.427050
std       45.976234      10.071733     28.756860      1.481543      2.792405
min        2.000000       0.000000      0.000000      0.000000      0.000000
25%       32.000000       0.000000     18.000000      1.000000      0.000000
50%       47.000000       0.000000     31.000000      2.000000      0.000000
75%       79.000000      15.000000     48.000000      3.000000      1.000000
max     1517.000000     236.000000    389.000000     42.000000     35.000000

       underscore_count   slash_count  question_count  equals_count  \
count      40000.000000  40000.000000    40000.000000  40000.000000
mean           0.402775      3.008100        0.240400      0.726825
std            1.107758      1.902334        0.451401      1.664428
min            0.000000      0.000000        0.000000      0.000000
25%            0.000000      2.000000        0.000000      0.000000
50%            0.000000      3.000000        0.000000      0.000000
75%            0.000000      4.000000        0.000000      0.000000
max           42.000000     22.000000       11.000000     24.000000

        digit_count   alpha_count      at_count  percent_count  \
```

```
count  40000.000000  40000.000000  40000.000000   40000.000000
mean       5.321325     46.065250      0.002575       0.346925
std       12.409322     32.698408      0.054483       3.900782
min        0.000000      2.000000      0.000000       0.000000
25%        0.000000     25.000000      0.000000       0.000000
50%        2.000000     37.000000      0.000000       0.000000
75%        6.000000     60.000000      0.000000       0.000000
max     1054.000000    840.000000      3.000000     147.000000

       ampersand_count       entropy    https_flag  login_keyword  \
count     40000.000000  40000.000000  40000.000000   40000.000000
mean          0.482225      4.218771      0.034425       0.032200
std           1.332490      0.425088      0.182320       0.176533
min           0.000000      1.000000      0.000000       0.000000
25%           0.000000      3.958229      0.000000       0.000000
50%           0.000000      4.224119      0.000000       0.000000
75%           0.000000      4.502583      0.000000       0.000000
max          22.000000      7.427747      1.000000       1.000000

       verify_keyword  secure_keyword  account_keyword
count    40000.000000    40000.000000     40000.000000
mean         0.002375        0.007150         0.009450
std          0.048677        0.084256         0.096752
min          0.000000        0.000000         0.000000
25%          0.000000        0.000000         0.000000
50%          0.000000        0.000000         0.000000
75%          0.000000        0.000000         0.000000
max          1.000000        1.000000         1.000000
```

```
# ============================================================
# SECTION 48: BUILD HYBRID FEATURE VECTORS (768 + 20 = 788)
# ============================================================
X_train_hybrid = np.concatenate([X_train_bert, X_train_url], axis=1)
X_test_hybrid = np.concatenate([X_test_bert, X_test_url], axis=1)

print("X_train_hybrid shape:", X_train_hybrid.shape)
print("X_test_hybrid shape :", X_test_hybrid.shape)

np.save("X_train_hybrid.npy", X_train_hybrid)
np.save("X_test_hybrid.npy", X_test_hybrid)
print("Hybrid feature vectors saved.")
```

```
X_train_hybrid shape: (40000, 788)
X_test_hybrid shape : (10000, 788)
Hybrid feature vectors saved.
```

```
# ============================================================
# SECTION 49: HYBRID FEATURES + SVM
# ============================================================
hybrid_svm = SVC(kernel="rbf", C=1.0, gamma="scale", probability=True, random_state=RANDOM_SEED)
hybrid_svm.fit(X_train_hybrid, train_labels_arr)

hybrid_svm_preds = hybrid_svm.predict(X_test_hybrid)
hybrid_svm_probs = hybrid_svm.predict_proba(X_test_hybrid)[:, 1]

hybrid_svm_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, hybrid_svm_preds),
    "Precision": precision_score(test_labels_arr, hybrid_svm_preds),
    "Recall": recall_score(test_labels_arr, hybrid_svm_preds),
    "F1": f1_score(test_labels_arr, hybrid_svm_preds)
}
print("Hybrid + SVM:", hybrid_svm_metrics)
print(classification_report(test_labels_arr, hybrid_svm_preds, target_names=["Safe","Unsafe"]))
cm_hybrid_svm = confusion_matrix(test_labels_arr, hybrid_svm_preds)
print(cm_hybrid_svm)

np.save("hybrid_svm_predictions.npy", hybrid_svm_preds)
np.save("hybrid_svm_probs.npy", hybrid_svm_probs)
```

```
Hybrid + SVM: {'Accuracy': 0.979, 'Precision': 0.978617106314948, 'Recall': 0.9794, 'F1': 0.9790083966413434}
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000
```

```
[[4893  107]
 [ 103 4897]]
```

```
# ============================================================
# SECTION 50: HYBRID FEATURES + XGBOOST
# ============================================================
hybrid_xgb = XGBClassifier(
    n_estimators=200, max_depth=6, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8,
    random_state=RANDOM_SEED, eval_metric="logloss"
)
hybrid_xgb.fit(X_train_hybrid, train_labels_arr)

hybrid_xgb_preds = hybrid_xgb.predict(X_test_hybrid)
hybrid_xgb_probs = hybrid_xgb.predict_proba(X_test_hybrid)[:, 1]

hybrid_xgb_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, hybrid_xgb_preds),
    "Precision": precision_score(test_labels_arr, hybrid_xgb_preds),
    "Recall": recall_score(test_labels_arr, hybrid_xgb_preds),
    "F1": f1_score(test_labels_arr, hybrid_xgb_preds)
}
print("Hybrid + XGBoost:", hybrid_xgb_metrics)
print(classification_report(test_labels_arr, hybrid_xgb_preds, target_names=["Safe","Unsafe"]))
cm_hybrid_xgb = confusion_matrix(test_labels_arr, hybrid_xgb_preds)
print(cm_hybrid_xgb)

np.save("hybrid_xgb_predictions.npy", hybrid_xgb_preds)
np.save("hybrid_xgb_probs.npy", hybrid_xgb_probs)
```

```
Hybrid + XGBoost: {'Accuracy': 0.98, 'Precision': 0.9807692307692307, 'Recall': 0.9792, 'F1': 0.9799839871897518}
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000

[[4904   96]
 [ 104 4896]]
```

```
# ============================================================
# SECTION 51: HYBRID FEATURES + RANDOM FOREST
# ============================================================
hybrid_rf = RandomForestClassifier(n_estimators=200, random_state=RANDOM_SEED, n_jobs=-1)
hybrid_rf.fit(X_train_hybrid, train_labels_arr)

hybrid_rf_preds = hybrid_rf.predict(X_test_hybrid)
hybrid_rf_probs = hybrid_rf.predict_proba(X_test_hybrid)[:, 1]

hybrid_rf_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, hybrid_rf_preds),
    "Precision": precision_score(test_labels_arr, hybrid_rf_preds),
    "Recall": recall_score(test_labels_arr, hybrid_rf_preds),
    "F1": f1_score(test_labels_arr, hybrid_rf_preds)
}
print("Hybrid + Random Forest:", hybrid_rf_metrics)
print(classification_report(test_labels_arr, hybrid_rf_preds, target_names=["Safe","Unsafe"]))
cm_hybrid_rf = confusion_matrix(test_labels_arr, hybrid_rf_preds)
print(cm_hybrid_rf)

np.save("hybrid_rf_predictions.npy", hybrid_rf_preds)
np.save("hybrid_rf_probs.npy", hybrid_rf_probs)
```

```
Hybrid + Random Forest: {'Accuracy': 0.9793, 'Precision': 0.9797797797797798, 'Recall': 0.9788, 'F1': 0.9792896448224112}
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000

[[4899  101]
 [ 106 4894]]
```

```
# ============================================================
# SECTION 52: FIGURE - HYBRID CLASSIFIERS CONFUSION MATRICES
# ============================================================
fig, axes = plt.subplots(1, 3, figsize=(18,5))

for ax, cm, title in zip(
    axes,
    [cm_hybrid_svm, cm_hybrid_xgb, cm_hybrid_rf],
    ["Hybrid + SVM", "Hybrid + XGBoost", "Hybrid + Random Forest"]
):
    sns.heatmap(cm, annot=True, fmt="d", cmap="Greens",
                xticklabels=["Safe","Unsafe"], yticklabels=["Safe","Unsafe"], ax=ax)
    ax.set_title(title)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.tight_layout()
plt.savefig("Figure18_Hybrid_Classifiers_Confusion_Matrices.png", dpi=300)
plt.show()

# ============================================================
# SECTION 53: FIGURE - ROC CURVE COMPARISON (HYBRID)
# ============================================================
plt.figure(figsize=(7,7))
for probs, label in zip(
    [hybrid_svm_probs, hybrid_xgb_probs, hybrid_rf_probs],
    ["Hybrid+SVM", "Hybrid+XGBoost", "Hybrid+RF"]
):
    fpr, tpr, _ = roc_curve(test_labels_arr, probs)
    roc_auc_val = auc(fpr, tpr)
    plt.plot(fpr, tpr, linewidth=2, label=f"{label} (AUC={roc_auc_val:.4f})")

plt.plot([0,1],[0,1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Figure 19: ROC Curve Comparison — Hybrid Features + Classical Classifiers")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Figure19_Hybrid_Classifiers_ROC_Comparison.png", dpi=300)
plt.show()
```

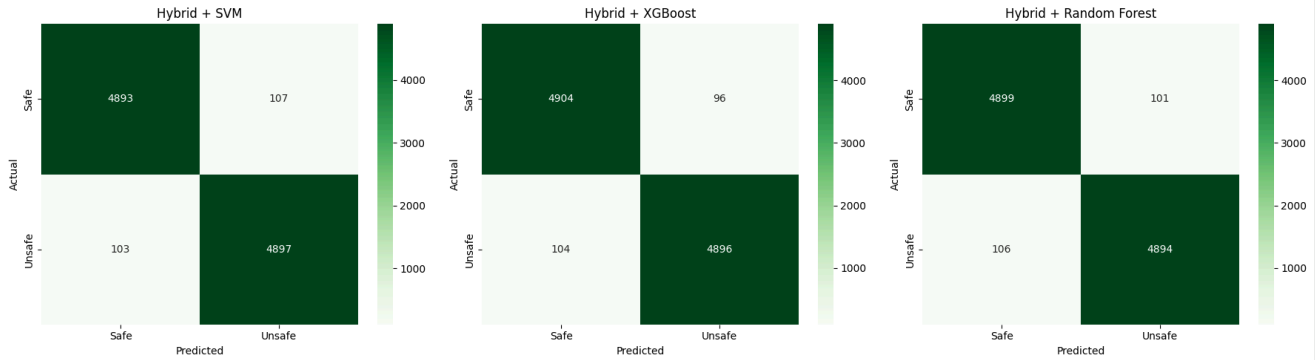
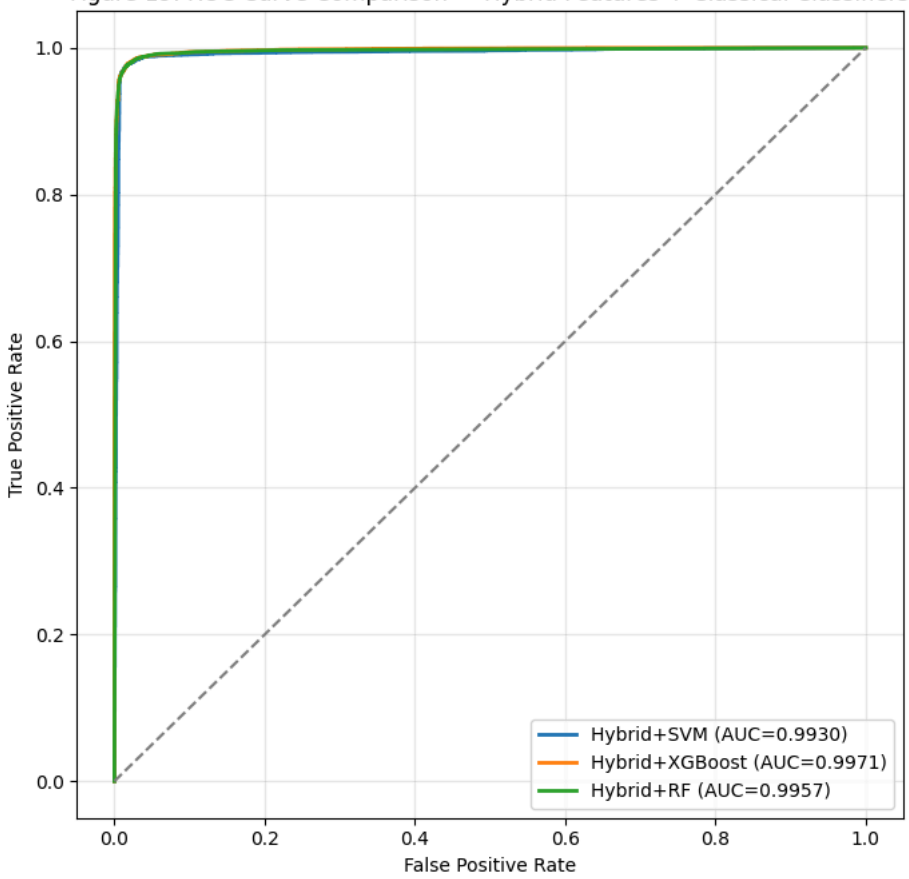


Figure 19: ROC Curve Comparison — Hybrid Features + Classical Classifiers




```
# ============================================================
# SECTION 54: PROPOSED MODEL — STEP 1: TRAIN XGBOOST (LEAF GENERATOR)
# ============================================================
xgb_leaf_model = XGBClassifier(
    n_estimators=200, max_depth=6, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8,
    random_state=RANDOM_SEED, eval_metric="logloss"
)
xgb_leaf_model.fit(X_train_hybrid, train_labels_arr)

print("XGBoost leaf-generating model trained.")
```

```
XGBoost leaf-generating model trained.
```

```
# ============================================================
# SECTION 55: PROPOSED MODEL — STEP 2: EXTRACT LEAF-NODE INDICES
# ============================================================
train_leaf_indices = xgb_leaf_model.apply(X_train_hybrid)
test_leaf_indices = xgb_leaf_model.apply(X_test_hybrid)

print("Train leaf indices shape:", train_leaf_indices.shape)
print("Test leaf indices shape :", test_leaf_indices.shape)
```

```
Train leaf indices shape: (40000, 200)
Test leaf indices shape : (10000, 200)
```

```
# ============================================================
# SECTION 56: PROPOSED MODEL — STEP 3: BUILD FINAL FEATURE SET
# Hybrid (788) + XGBoost Leaf Indices (200) = 988-dim
# ============================================================
X_train_final = np.concatenate([X_train_hybrid, train_leaf_indices], axis=1)
X_test_final = np.concatenate([X_test_hybrid, test_leaf_indices], axis=1)

print("X_train_final shape:", X_train_final.shape)
print("X_test_final shape :", X_test_final.shape)

np.save("X_train_final_proposed.npy", X_train_final)
np.save("X_test_final_proposed.npy", X_test_final)
```

```
X_train_final shape: (40000, 988)
X_test_final shape : (10000, 988)
```

```
# ============================================================
# SECTION 57: PROPOSED MODEL — STEP 4: TRAIN FINAL RANDOM FOREST
# ============================================================
proposed_rf = RandomForestClassifier(n_estimators=200, random_state=RANDOM_SEED, n_jobs=-1)
proposed_rf.fit(X_train_final, train_labels_arr)

print("Proposed model (Hybrid + XGBoost-Leaf + Random Forest) trained.")
```

```
Proposed model (Hybrid + XGBoost-Leaf + Random Forest) trained.
```

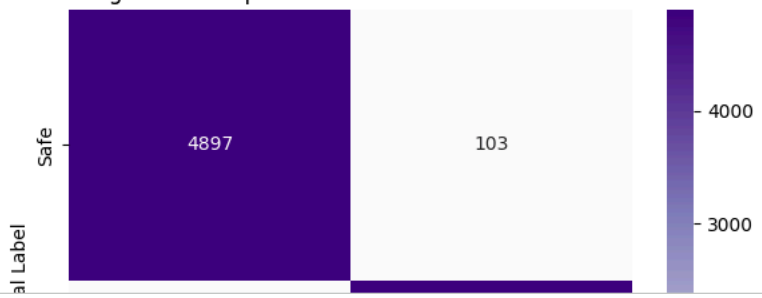
```
# ============================================================
# SECTION 58: PROPOSED MODEL — STEP 5: EVALUATION
# ============================================================
proposed_preds = proposed_rf.predict(X_test_final)
proposed_probs = proposed_rf.predict_proba(X_test_final)[:, 1]

proposed_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, proposed_preds),
    "Precision": precision_score(test_labels_arr, proposed_preds),
    "Recall": recall_score(test_labels_arr, proposed_preds),
    "F1": f1_score(test_labels_arr, proposed_preds)
}
print("Proposed Model:", proposed_metrics)
print(classification_report(test_labels_arr, proposed_preds, target_names=["Safe","Unsafe"]))

cm_proposed = confusion_matrix(test_labels_arr, proposed_preds)
print(cm_proposed)

np.save("proposed_predictions.npy", proposed_preds)
np.save("proposed_probs.npy", proposed_probs)
```

```
Proposed Model: {'Accuracy': 0.9793, 'Precision': 0.9793958791758351, 'Recall': 0.9792, 'F1': 0.9792979297929792}
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000

[[4897  103]
 [ 104 4896]]
```

```
# ============================================================
# SECTION 59: FIGURE - PROPOSED MODEL CONFUSION MATRIX
# ============================================================
plt.figure(figsize=(6,5))
sns.heatmap(cm_proposed, annot=True, fmt="d", cmap="Purples",
            xticklabels=["Safe","Unsafe"], yticklabels=["Safe","Unsafe"])
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Figure 20: Proposed Model — Confusion Matrix")
plt.tight_layout()
plt.savefig("Figure20_Proposed_Confusion_Matrix.png", dpi=300)
plt.show()
```

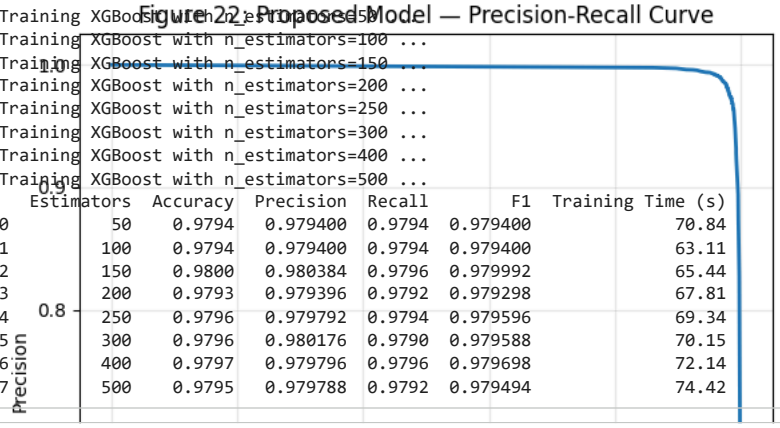
```
# ============================================================
# SECTION 60: FIGURE - PROPOSED MODEL ROC CURVE
# ============================================================
fpr_p, tpr_p, _ = roc_curve(test_labels_arr, proposed_probs)
roc_auc_p = auc(fpr_p, tpr_p)

plt.figure(figsize=(6,6))
plt.plot(fpr_p, tpr_p, linewidth=2, label=f"Proposed Model (AUC={roc_auc_p:.4f})")
plt.plot([0,1],[0,1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Figure 21: Proposed Model — ROC Curve")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Figure21_Proposed_ROC_Curve.png", dpi=300)
plt.show()

# ============================================================
# SECTION 61: FIGURE - PROPOSED MODEL PRECISION-RECALL CURVE
# ============================================================
prec_p, rec_p, _ = precision_recall_curve(test_labels_arr, proposed_probs)
avg_prec_p = average_precision_score(test_labels_arr, proposed_probs)

plt.figure(figsize=(6,6))
plt.plot(rec_p, prec_p, linewidth=2, label=f"Proposed Model (AP={avg_prec_p:.4f})")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Figure 22: Proposed Model — Precision-Recall Curve")
plt.legend(loc="lower left")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Figure22_Proposed_PR_Curve.png", dpi=300)
plt.show()

print("Proposed Model ROC-AUC:", roc_auc_p)
print("Proposed Model Average Precision:", avg_prec_p)
```

Figure 20: Proposed Model — Confusion Matrix




```
# ============================================================
# SECTION 62: HYPERPARAMETER TUNING — XGBOOST N_ESTIMATORS
# ============================================================
import time

tuning_results_estimators = []
estimator_values = [50, 100, 150, 200, 250, 300, 400, 500]

for n in estimator_values:
    print(f"Training XGBoost with n_estimators={n} ...")
    start = time.time()

    xgb_tune = XGBClassifier(
        n_estimators=n, max_depth=6, learning_rate=0.1,
        subsample=0.8, colsample_bytree=0.8,
        random_state=RANDOM_SEED, eval_metric="logloss"
    )
    xgb_tune.fit(X_train_hybrid, train_labels_arr)

    train_leaf = xgb_tune.apply(X_train_hybrid)
    test_leaf = xgb_tune.apply(X_test_hybrid)

    X_train_temp = np.concatenate([X_train_hybrid, train_leaf], axis=1)
    X_test_temp = np.concatenate([X_test_hybrid, test_leaf], axis=1)

    rf_tune = RandomForestClassifier(n_estimators=200, random_state=RANDOM_SEED, n_jobs=-1)
    rf_tune.fit(X_train_temp, train_labels_arr)
    preds_tune = rf_tune.predict(X_test_temp)

    end = time.time()

    tuning_results_estimators.append({
        "Estimators": n,
        "Accuracy": accuracy_score(test_labels_arr, preds_tune),
        "Precision": precision_score(test_labels_arr, preds_tune),
        "Recall": recall_score(test_labels_arr, preds_tune),
        "F1": f1_score(test_labels_arr, preds_tune),
        "Training Time (s)": round(end - start, 2)
    })

results_estimators_df = pd.DataFrame(tuning_results_estimators)
results_estimators_df.to_csv("Tuning_XGBoost_Estimators.csv", index=False)
print(results_estimators_df)
```

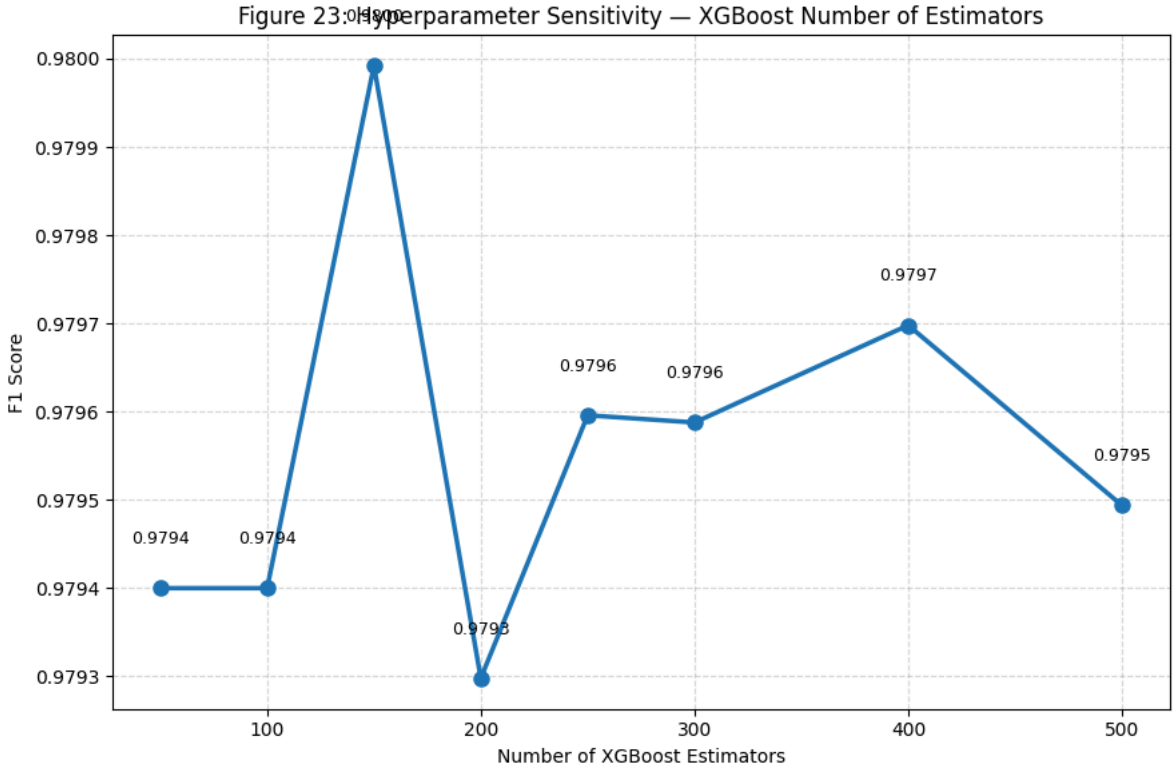
Figure 22: Proposed Model — Precision-Recall Curve




```
Training XGBoost with n_estimators=50 ...
Training XGBoost with n_estimators=100 ...
Training XGBoost with n_estimators=150 ...
Training XGBoost with n_estimators=200 ...
Training XGBoost with n_estimators=250 ...
Training XGBoost with n_estimators=300 ...
Training XGBoost with n_estimators=400 ...
Training XGBoost with n_estimators=500 ...
   Estimators  Accuracy  Precision  Recall        F1  Training Time (s)
0          50    0.9794   0.979400  0.9794  0.979400              70.84
1         100    0.9794   0.979400  0.9794  0.979400              63.11
2         150    0.9800   0.980384  0.9796  0.979992              65.44
3         200    0.9793   0.979396  0.9792  0.979298              67.81
4         250    0.9796   0.979792  0.9794  0.979596              69.34
5         300    0.9796   0.980176  0.9790  0.979588              70.15
6         400    0.9797   0.979796  0.9796  0.979698              72.14
7         500    0.9795   0.979788  0.9792  0.979494              74.42
```

```
# ============================================================
# SECTION 63: FIGURE - XGBOOST N_ESTIMATORS SENSITIVITY
```

```
# ============================================================
plt.figure(figsize=(9,6))
plt.plot(results_estimators_df["Estimators"], results_estimators_df["F1"],
         marker="o", linewidth=2.5, markersize=8)
for x, y in zip(results_estimators_df["Estimators"], results_estimators_df["F1"]):
    plt.text(x, y + 0.00005, f"{y:.4f}", ha="center", fontsize=9)
plt.xlabel("Number of XGBoost Estimators")
plt.ylabel("F1 Score")
plt.title("Figure 23: Hyperparameter Sensitivity — XGBoost Number of Estimators")
plt.grid(linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("Figure23_XGBoost_Estimators_Tuning.png", dpi=300)
plt.show()

best_est = results_estimators_df.loc[results_estimators_df["F1"].idxmax()]
print("Best n_estimators configuration:")
print(best_est)
```



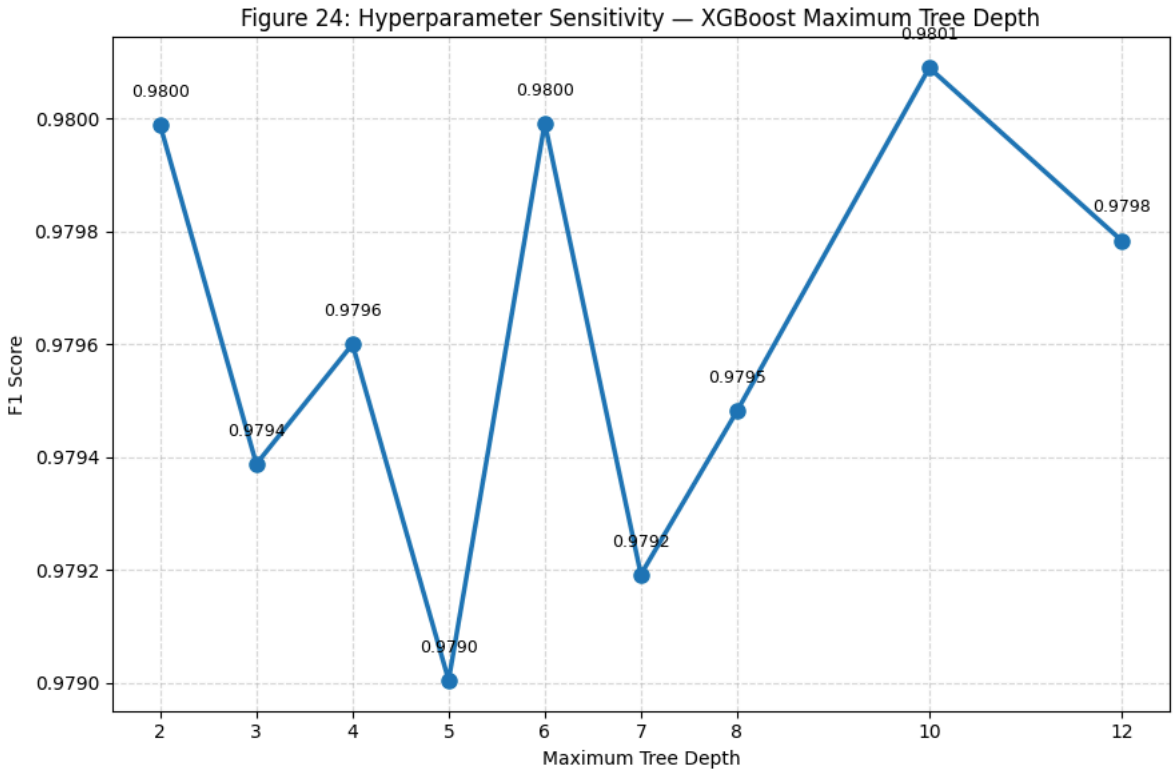

```
Best n_estimators configuration:
Estimators           150.000000
Accuracy               0.980000
Precision              0.980384
Recall                 0.979600
F1                     0.979992
Training Time (s)     65.440000
Name: 2, dtype: float64
```

```
# ============================================================
# SECTION 64: HYPERPARAMETER TUNING — XGBOOST MAX_DEPTH
# ============================================================
tuning_results_depth = []
depth_values = [2, 3, 4, 5, 6, 7, 8, 10, 12]

BEST_N_ESTIMATORS = int(best_est["Estimators"])  # carried from previous sweep

for depth in depth_values:
    print(f"Training XGBoost with max_depth={depth} ...")
    start = time.time()

    xgb_tune = XGBClassifier(
        n_estimators=BEST_N_ESTIMATORS, max_depth=depth, learning_rate=0.1,
        subsample=0.8, colsample_bytree=0.8,
        random_state=RANDOM_SEED, eval_metric="logloss"
    )
    xgb_tune.fit(X_train_hybrid, train_labels_arr)
```

```
    train_leaf = xgb_tune.apply(X_train_hybrid)
    test_leaf = xgb_tune.apply(X_test_hybrid)

    X_train_temp = np.concatenate([X_train_hybrid, train_leaf], axis=1)
    X_test_temp = np.concatenate([X_test_hybrid, test_leaf], axis=1)

    rf_tune = RandomForestClassifier(n_estimators=200, random_state=RANDOM_SEED, n_jobs=-1)
    rf_tune.fit(X_train_temp, train_labels_arr)
    preds_tune = rf_tune.predict(X_test_temp)

    end = time.time()

    tuning_results_depth.append({
        "Max Depth": depth,
        "Accuracy": accuracy_score(test_labels_arr, preds_tune),
        "Precision": precision_score(test_labels_arr, preds_tune),
        "Recall": recall_score(test_labels_arr, preds_tune),
        "F1": f1_score(test_labels_arr, preds_tune),
        "Training Time (s)": round(end - start, 2)
    })

results_depth_df = pd.DataFrame(tuning_results_depth)
results_depth_df.to_csv("Tuning_XGBoost_MaxDepth.csv", index=False)
print(results_depth_df)
```

```
Training XGBoost with max_depth=2 ...
Training XGBoost with max_depth=3 ...
Training XGBoost with max_depth=4 ...
Training XGBoost with max_depth=5 ...
Training XGBoost with max_depth=6 ...
Training XGBoost with max_depth=7 ...
Training XGBoost with max_depth=8 ...
Training XGBoost with max_depth=10 ...
Training XGBoost with max_depth=12 ...
   Max Depth  Accuracy  Precision  Recall        F1  Training Time (s)
0          2    0.9800   0.980577  0.9794  0.979988              71.33
1          3    0.9794   0.979976  0.9788  0.979388              71.78
2          4    0.9796   0.979600  0.9796  0.979600              70.25
3          5    0.9790   0.978808  0.9792  0.979004              68.56
4          6    0.9800   0.980384  0.9796  0.979992              66.99
5          7    0.9792   0.979584  0.9788  0.979192              61.51
6          8    0.9795   0.980365  0.9786  0.979482              58.01
7         10    0.9801   0.980581  0.9796  0.980090              55.48
8         12    0.9798   0.980569  0.9790  0.979784              55.33
```

```
# ============================================================
# SECTION 65: FIGURE - XGBOOST MAX_DEPTH SENSITIVITY
# ============================================================
plt.figure(figsize=(9,6))
plt.plot(results_depth_df["Max Depth"], results_depth_df["F1"],
         marker="o", linewidth=2.5, markersize=8)
for x, y in zip(results_depth_df["Max Depth"], results_depth_df["F1"]):
    plt.text(x, y + 0.00005, f"{y:.4f}", ha="center", fontsize=9)
plt.xlabel("Maximum Tree Depth")
plt.ylabel("F1 Score")
plt.title("Figure 24: Hyperparameter Sensitivity — XGBoost Maximum Tree Depth")
plt.xticks(results_depth_df["Max Depth"])
plt.grid(linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("Figure24_XGBoost_MaxDepth_Tuning.png", dpi=300)
plt.show()

best_depth = results_depth_df.loc[results_depth_df["F1"].idxmax()]
print("Best max_depth configuration:")
print(best_depth)
```



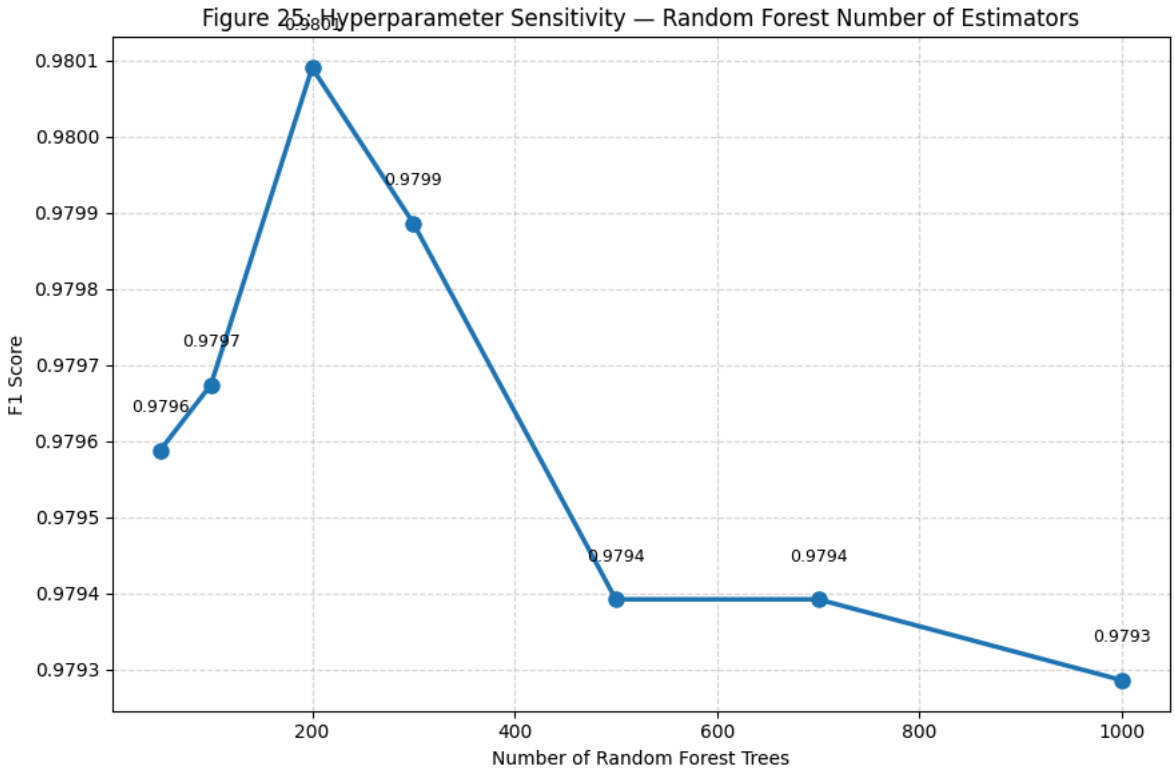

```
Best max_depth configuration:
Max Depth            10.000000
Accuracy              0.980100
Precision             0.980581
Recall                0.979600
F1                    0.980090
Training Time (s)    55.480000
Name: 7, dtype: float64
```

```
# ============================================================
# SECTION 66: HYPERPARAMETER TUNING — RANDOM FOREST N_ESTIMATORS
# ============================================================
tuning_results_rf = []
rf_estimator_values = [50, 100, 200, 300, 500, 700, 1000]

BEST_MAX_DEPTH = int(best_depth["Max Depth"])  # carried from previous sweep

# Build the optimized XGBoost leaf features once (fixed across this sweep)
xgb_optimized = XGBClassifier(
    n_estimators=BEST_N_ESTIMATORS, max_depth=BEST_MAX_DEPTH, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8,
    random_state=RANDOM_SEED, eval_metric="logloss"
)
xgb_optimized.fit(X_train_hybrid, train_labels_arr)

train_leaf_opt = xgb_optimized.apply(X_train_hybrid)
test_leaf_opt = xgb_optimized.apply(X_test_hybrid)

X_train_opt = np.concatenate([X_train_hybrid, train_leaf_opt], axis=1)
X_test_opt = np.concatenate([X_test_hybrid, test_leaf_opt], axis=1)

print("Optimized XGBoost leaf features built. Shape:", X_train_opt.shape)

for n in rf_estimator_values:
    print(f"Training Random Forest with n_estimators={n} ...")
    start = time.time()

    rf_tune = RandomForestClassifier(n_estimators=n, random_state=RANDOM_SEED, n_jobs=-1)
    rf_tune.fit(X_train_opt, train_labels_arr)
    preds_tune = rf_tune.predict(X_test_opt)

    end = time.time()

    tuning_results_rf.append({
        "RF Estimators": n,
```

```
        "Accuracy": accuracy_score(test_labels_arr, preds_tune),
        "Precision": precision_score(test_labels_arr, preds_tune),
        "Recall": recall_score(test_labels_arr, preds_tune),
        "F1": f1_score(test_labels_arr, preds_tune),
        "Training Time (s)": round(end - start, 2)
    })

results_rf_df = pd.DataFrame(tuning_results_rf)
results_rf_df.to_csv("Tuning_RF_Estimators.csv", index=False)
print(results_rf_df)
```

```
Optimized XGBoost leaf features built. Shape: (40000, 938)
Training Random Forest with n_estimators=50 ...
Training Random Forest with n_estimators=100 ...
Training Random Forest with n_estimators=200 ...
Training Random Forest with n_estimators=300 ...
Training Random Forest with n_estimators=500 ...
Training Random Forest with n_estimators=700 ...
Training Random Forest with n_estimators=1000 ...
   RF Estimators  Accuracy  Precision  Recall        F1  Training Time (s)
0             50    0.9796   0.980176  0.9790  0.979588              12.08
1            100    0.9797   0.980950  0.9784  0.979674              21.53
2            200    0.9801   0.980581  0.9796  0.980090              42.05
3            300    0.9799   0.980573  0.9792  0.979886              62.66
4            500    0.9794   0.979784  0.9790  0.979392             103.55
5            700    0.9794   0.979784  0.9790  0.979392             146.13
6           1000    0.9793   0.979972  0.9786  0.979285             210.16
```

```
# ============================================================
# SECTION 67: FIGURE - RANDOM FOREST N_ESTIMATORS SENSITIVITY
# ============================================================
plt.figure(figsize=(9,6))
plt.plot(results_rf_df["RF Estimators"], results_rf_df["F1"],
         marker="o", linewidth=2.5, markersize=8)
for x, y in zip(results_rf_df["RF Estimators"], results_rf_df["F1"]):
    plt.text(x, y + 0.00005, f"{y:.4f}", ha="center", fontsize=9)
plt.xlabel("Number of Random Forest Trees")
plt.ylabel("F1 Score")
plt.title("Figure 25: Hyperparameter Sensitivity — Random Forest Number of Estimators")
plt.grid(linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("Figure25_RF_Estimators_Tuning.png", dpi=300)
plt.show()

best_rf = results_rf_df.loc[results_rf_df["F1"].idxmax()]
print("Best RF n_estimators configuration:")
print(best_rf)

print("\n=== FINAL OPTIMAL CONFIGURATION ===")
print("XGBoost n_estimators:", BEST_N_ESTIMATORS)
print("XGBoost max_depth   :", BEST_MAX_DEPTH)
print("Random Forest n_estimators:", int(best_rf["RF Estimators"]))
```




```
Best RF n_estimators configuration:
RF Estimators        200.000000
Accuracy               0.980100
Precision              0.980581
Recall                 0.979600
F1                     0.980090
Training Time (s)     42.050000
Name: 2, dtype: float64

=== FINAL OPTIMAL CONFIGURATION ===
XGBoost n_estimators: 150
XGBoost max_depth   : 10
Random Forest n_estimators: 200
```

```
# ============================================================
# SECTION 68: FINAL OPTIMIZED MODEL — STEP 1: XGBOOST (TUNED)
# ============================================================
FINAL_XGB_ESTIMATORS = 150
FINAL_XGB_MAX_DEPTH = 10
FINAL_RF_ESTIMATORS = 200

xgb_final_optimized = XGBClassifier(
    n_estimators=FINAL_XGB_ESTIMATORS,
    max_depth=FINAL_XGB_MAX_DEPTH,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=RANDOM_SEED,
    eval_metric="logloss"
)
xgb_final_optimized.fit(X_train_hybrid, train_labels_arr)

print("Final optimized XGBoost leaf-generator trained.")
print(f"Config: n_estimators={FINAL_XGB_ESTIMATORS}, max_depth={FINAL_XGB_MAX_DEPTH}")
```

```
Final optimized XGBoost leaf-generator trained.
Config: n_estimators=150, max_depth=10
```

```
# ============================================================
# SECTION 69: FINAL OPTIMIZED MODEL — STEP 2: LEAF FEATURES
# ============================================================
train_leaf_final = xgb_final_optimized.apply(X_train_hybrid)
test_leaf_final = xgb_final_optimized.apply(X_test_hybrid)

X_train_final_opt = np.concatenate([X_train_hybrid, train_leaf_final], axis=1)
X_test_final_opt = np.concatenate([X_test_hybrid, test_leaf_final], axis=1)
```

```
print("X_train_final_opt shape:", X_train_final_opt.shape)
print("X_test_final_opt shape :", X_test_final_opt.shape)

np.save("X_train_final_optimized.npy", X_train_final_opt)
np.save("X_test_final_optimized.npy", X_test_final_opt)
```

```
X_train_final_opt shape: (40000, 938)
X_test_final_opt shape : (10000, 938)
```

```
# ============================================================
# SECTION 70: FINAL OPTIMIZED MODEL — STEP 3: FINAL RANDOM FOREST
# ============================================================
rf_final_optimized = RandomForestClassifier(
    n_estimators=FINAL_RF_ESTIMATORS,
    random_state=RANDOM_SEED,
    n_jobs=-1
)
rf_final_optimized.fit(X_train_final_opt, train_labels_arr)

print("Final optimized Random Forest trained.")
```

```
Final optimized Random Forest trained.
```

```
# ============================================================
# SECTION 71: FINAL OPTIMIZED MODEL — STEP 4: EVALUATION
# ============================================================
final_opt_preds = rf_final_optimized.predict(X_test_final_opt)
final_opt_probs = rf_final_optimized.predict_proba(X_test_final_opt)[:, 1]

final_opt_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, final_opt_preds),
    "Precision": precision_score(test_labels_arr, final_opt_preds),
    "Recall": recall_score(test_labels_arr, final_opt_preds),
    "F1": f1_score(test_labels_arr, final_opt_preds)
}
print("Final Optimized Proposed Model:", final_opt_metrics)
print(classification_report(test_labels_arr, final_opt_preds, target_names=["Safe","Unsafe"]))

cm_final_opt = confusion_matrix(test_labels_arr, final_opt_preds)
print(cm_final_opt)

roc_auc_final = auc(*roc_curve(test_labels_arr, final_opt_probs)[:2][::-1] if False else (lambda f,t,_: (f,t))(*roc_curve(test_
```

```
Final Optimized Proposed Model: {'Accuracy': 0.9801, 'Precision': 0.9805805805805806, 'Recall': 0.9796, 'F1': 0.9800900450225113
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000

[[4903   97]
 [ 102 4898]]
```

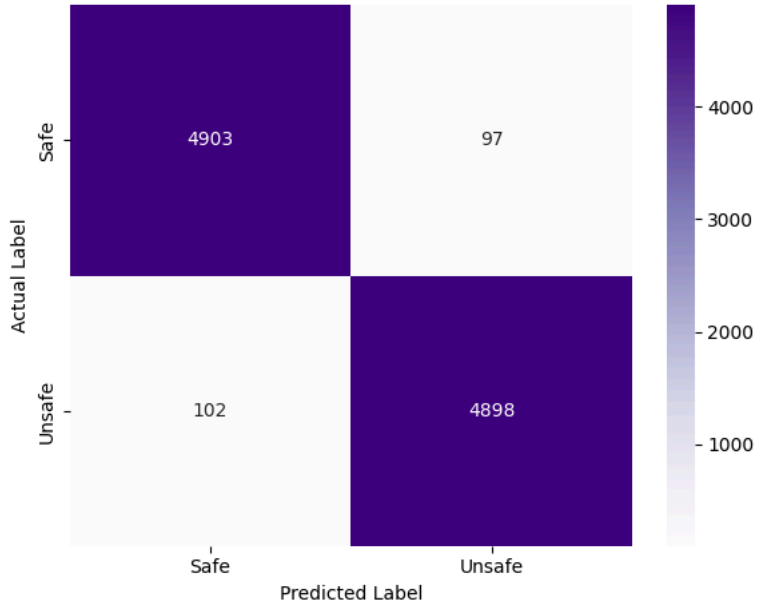
```
# ============================================================
# SECTION 71: FINAL OPTIMIZED MODEL — STEP 4: EVALUATION (CORRECTED)
# ============================================================
final_opt_preds = rf_final_optimized.predict(X_test_final_opt)
final_opt_probs = rf_final_optimized.predict_proba(X_test_final_opt)[:, 1]

final_opt_metrics = {
    "Accuracy": accuracy_score(test_labels_arr, final_opt_preds),
    "Precision": precision_score(test_labels_arr, final_opt_preds),
    "Recall": recall_score(test_labels_arr, final_opt_preds),
    "F1": f1_score(test_labels_arr, final_opt_preds)
}
print("Final Optimized Proposed Model:", final_opt_metrics)
print(classification_report(test_labels_arr, final_opt_preds, target_names=["Safe","Unsafe"]))

cm_final_opt = confusion_matrix(test_labels_arr, final_opt_preds)
print(cm_final_opt)

fpr_final, tpr_final, _ = roc_curve(test_labels_arr, final_opt_probs)
```

```
roc_auc_final = auc(fpr_final, tpr_final)

prec_final, rec_final, _ = precision_recall_curve(test_labels_arr, final_opt_probs)
avg_prec_final = average_precision_score(test_labels_arr, final_opt_probs)

print("Final Optimized Model ROC-AUC:", roc_auc_final)
print("Final Optimized Model Average Precision:", avg_prec_final)

np.save("final_optimized_predictions.npy", final_opt_preds)
np.save("final_optimized_probs.npy", final_opt_probs)
```

```
Final Optimized Proposed Model: {'Accuracy': 0.9801, 'Precision': 0.9805805805805806, 'Recall': 0.9796, 'F1': 0.9800900450225113
              precision    recall  f1-score   support

        Safe       0.98      0.98      0.98      5000
      Unsafe       0.98      0.98      0.98      5000

    accuracy                           0.98     10000
   macro avg       0.98      0.98      0.98     10000
weighted avg       0.98      0.98      0.98     10000

[[4903   97]
 [ 102 4898]]
Final Optimized Model ROC-AUC: 0.9952979399999999
Final Optimized Model Average Precision: 0.9943591371701103
```

```
# ============================================================
# SECTION 72: FIGURE - FINAL OPTIMIZED MODEL CONFUSION MATRIX
# ============================================================
plt.figure(figsize=(6,5))
sns.heatmap(cm_final_opt, annot=True, fmt="d", cmap="Purples",
            xticklabels=["Safe","Unsafe"], yticklabels=["Safe","Unsafe"])
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Figure 26: Final Optimized Proposed Model — Confusion Matrix")
plt.tight_layout()
plt.savefig("Figure26_Final_Optimized_Confusion_Matrix.png", dpi=300)
plt.show()

# ============================================================
# SECTION 73: SAVE FINAL MODEL ARTIFACTS
# ============================================================
import joblib
joblib.dump(xgb_final_optimized, "xgb_final_optimized_model.joblib")
joblib.dump(rf_final_optimized, "rf_final_optimized_model.joblib")

print("Final model artifacts saved:")
print(" - xgb_final_optimized_model.joblib")
print(" - rf_final_optimized_model.joblib")
```

Figure 26: Final Optimized Proposed Model — Confusion Matrix

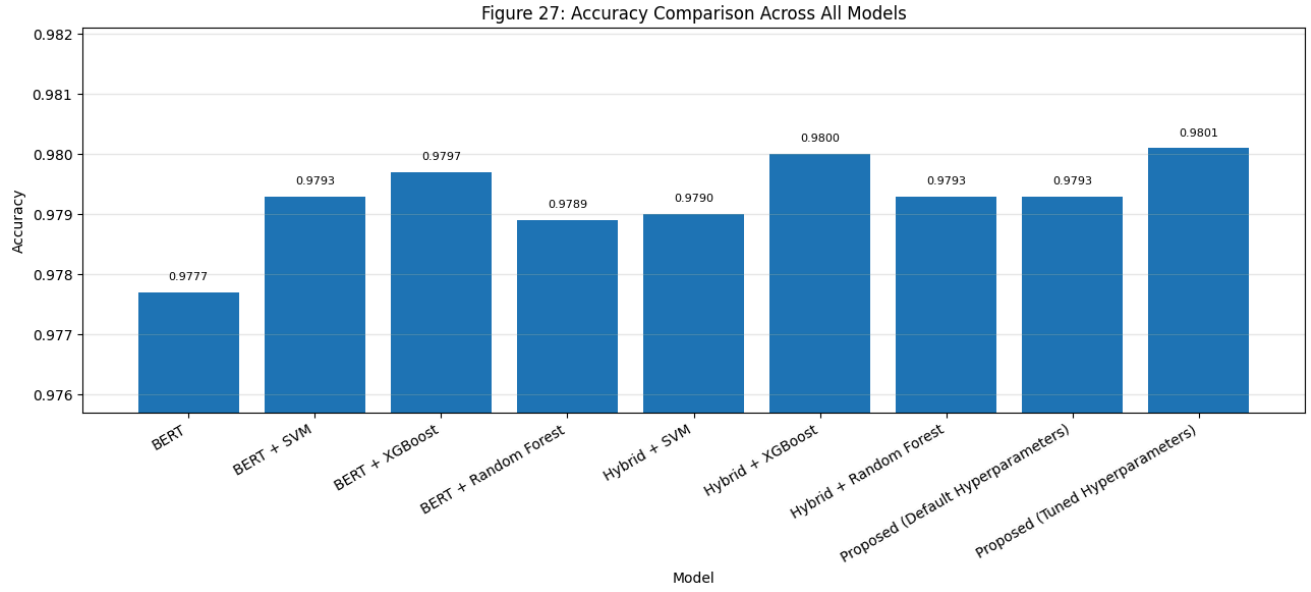



```
Final model artifacts saved:
 - xgb_final_optimized_model.joblib
 - rf_final_optimized_model.joblib
```

```
# ============================================================
# SECTION 74: MASTER MODEL COMPARISON TABLE
# ============================================================
comparison_data = [
    {"Model": "BERT",
     "Accuracy": 0.9777, "Precision": 0.97775, "Recall": 0.9777, "F1": 0.977699},

    {"Model": "BERT + SVM",
     "Accuracy": svm_metrics["Accuracy"], "Precision": svm_metrics["Precision"],
     "Recall": svm_metrics["Recall"], "F1": svm_metrics["F1"]},

    {"Model": "BERT + XGBoost",
     "Accuracy": xgb_bert_metrics["Accuracy"], "Precision": xgb_bert_metrics["Precision"],
     "Recall": xgb_bert_metrics["Recall"], "F1": xgb_bert_metrics["F1"]},

    {"Model": "BERT + Random Forest",
     "Accuracy": rf_bert_metrics["Accuracy"], "Precision": rf_bert_metrics["Precision"],
     "Recall": rf_bert_metrics["Recall"], "F1": rf_bert_metrics["F1"]},

    {"Model": "Hybrid + SVM",
     "Accuracy": hybrid_svm_metrics["Accuracy"], "Precision": hybrid_svm_metrics["Precision"],
     "Recall": hybrid_svm_metrics["Recall"], "F1": hybrid_svm_metrics["F1"]},

    {"Model": "Hybrid + XGBoost",
     "Accuracy": hybrid_xgb_metrics["Accuracy"], "Precision": hybrid_xgb_metrics["Precision"],
     "Recall": hybrid_xgb_metrics["Recall"], "F1": hybrid_xgb_metrics["F1"]},

    {"Model": "Hybrid + Random Forest",
     "Accuracy": hybrid_rf_metrics["Accuracy"], "Precision": hybrid_rf_metrics["Precision"],
     "Recall": hybrid_rf_metrics["Recall"], "F1": hybrid_rf_metrics["F1"]},

    {"Model": "Proposed (Default Hyperparameters)",
     "Accuracy": proposed_metrics["Accuracy"], "Precision": proposed_metrics["Precision"],
     "Recall": proposed_metrics["Recall"], "F1": proposed_metrics["F1"]},

    {"Model": "Proposed (Tuned Hyperparameters)",
     "Accuracy": final_opt_metrics["Accuracy"], "Precision": final_opt_metrics["Precision"],
     "Recall": final_opt_metrics["Recall"], "F1": final_opt_metrics["F1"]},
]

master_comparison_df = pd.DataFrame(comparison_data)
master_comparison_df = master_comparison_df.round(4)
master_comparison_df.to_csv("Master_Model_Comparison.csv", index=False)

print(master_comparison_df.to_string(index=False))
```

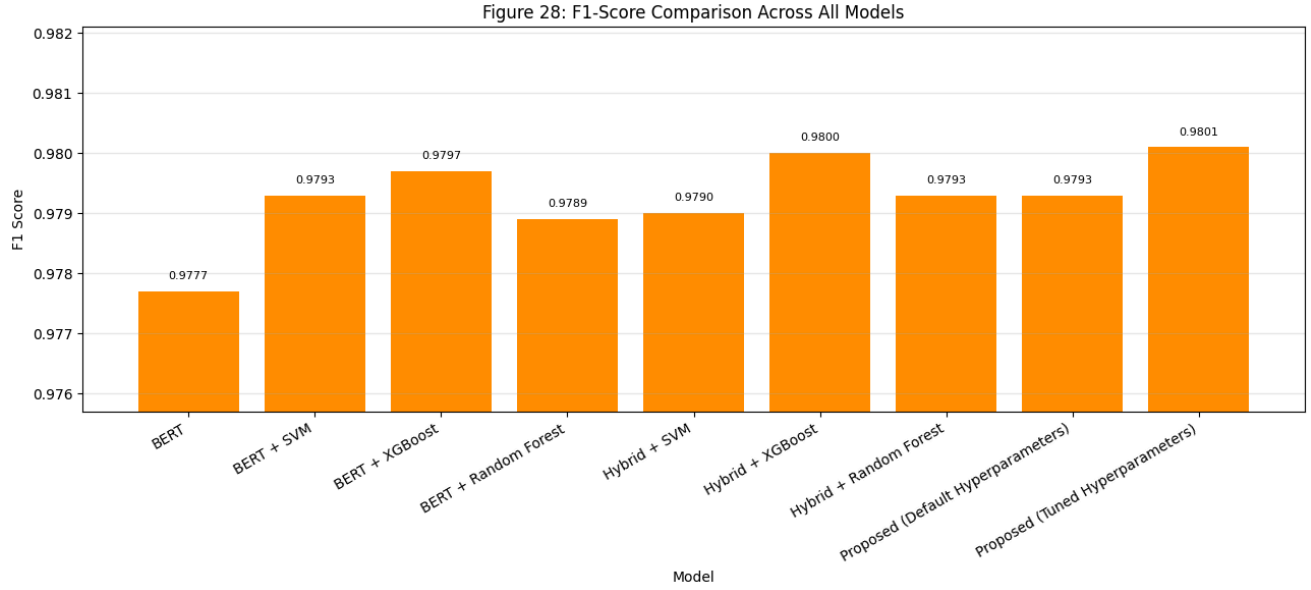
```
                            Model  Accuracy  Precision  Recall     F1
                             BERT    0.9777     0.9778  0.9777 0.9777
                        BERT + SVM    0.9793     0.9792  0.9794 0.9793
                    BERT + XGBoost    0.9797     0.9802  0.9792 0.9797
                BERT + Random Forest    0.9789     0.9788  0.9790 0.9789
                        Hybrid + SVM    0.9790     0.9786  0.9794 0.9790
                    Hybrid + XGBoost    0.9800     0.9808  0.9792 0.9800
                Hybrid + Random Forest    0.9793     0.9798  0.9788 0.9793
Proposed (Default Hyperparameters)    0.9793     0.9794  0.9792 0.9793
  Proposed (Tuned Hyperparameters)    0.9801     0.9806  0.9796 0.9801
```

```
# ============================================================
# SECTION 75: FIGURE - ACCURACY COMPARISON (ALL MODELS)
# ============================================================
plt.figure(figsize=(13,6))
bars = plt.bar(master_comparison_df["Model"], master_comparison_df["Accuracy"])
plt.ylabel("Accuracy")
plt.xlabel("Model")
plt.title("Figure 27: Accuracy Comparison Across All Models")
plt.xticks(rotation=30, ha="right")
plt.ylim(master_comparison_df["Accuracy"].min() - 0.002, master_comparison_df["Accuracy"].max() + 0.002)

for bar, val in zip(bars, master_comparison_df["Accuracy"]):
    plt.text(bar.get_x() + bar.get_width()/2, val + 0.0002, f"{val:.4f}", ha="center", fontsize=8)

plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("Figure27_Master_Accuracy_Comparison.png", dpi=300)
plt.show()
```



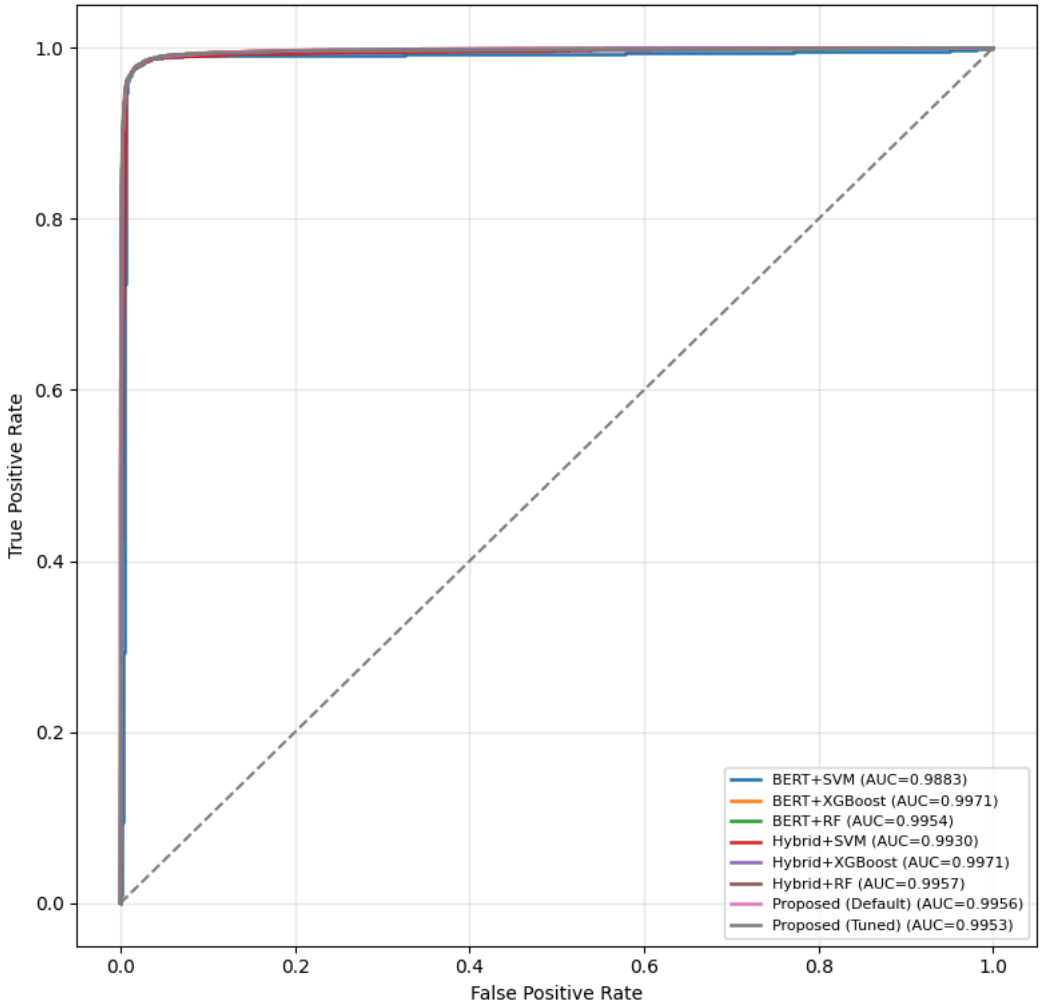

```
# ============================================================
# SECTION 76: FIGURE - F1 SCORE COMPARISON (ALL MODELS)
# ============================================================
plt.figure(figsize=(13,6))
bars = plt.bar(master_comparison_df["Model"], master_comparison_df["F1"], color="darkorange")
plt.ylabel("F1 Score")
plt.xlabel("Model")
plt.title("Figure 28: F1-Score Comparison Across All Models")
plt.xticks(rotation=30, ha="right")
plt.ylim(master_comparison_df["F1"].min() - 0.002, master_comparison_df["F1"].max() + 0.002)

for bar, val in zip(bars, master_comparison_df["F1"]):
    plt.text(bar.get_x() + bar.get_width()/2, val + 0.0002, f"{val:.4f}", ha="center", fontsize=8)

plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("Figure28_Master_F1_Comparison.png", dpi=300)
plt.show()
```

Figure 28: F1-Score Comparison Across All Models



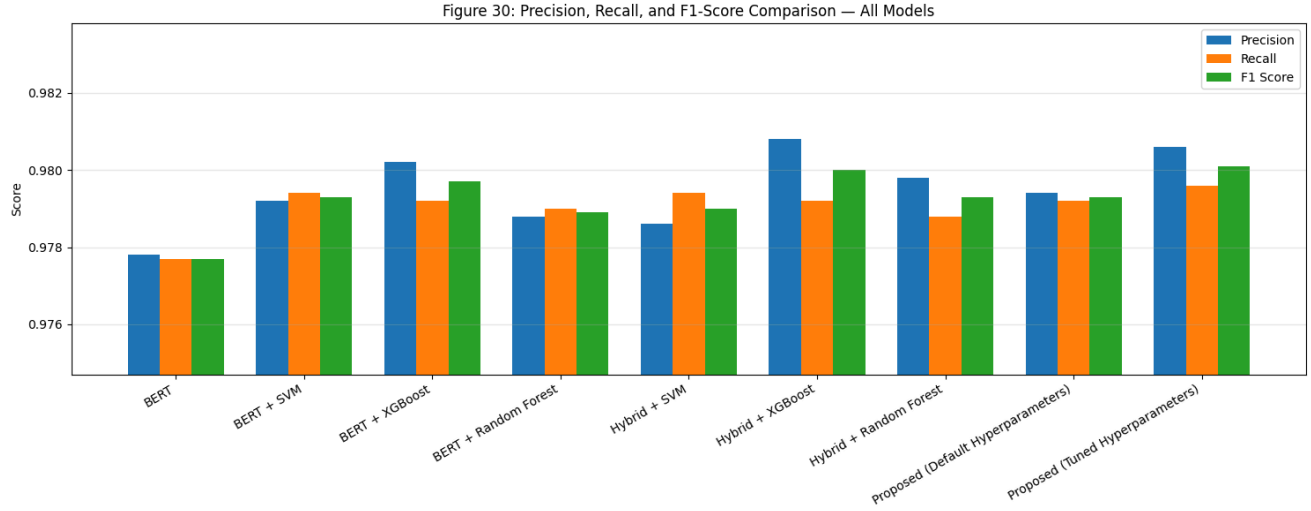

```
# ============================================================
# SECTION 77: FIGURE - ROC CURVE COMPARISON (ALL MODELS)
# ============================================================
plt.figure(figsize=(8,8))

roc_models = [
    (svm_probs, "BERT+SVM"),
    (xgb_bert_probs, "BERT+XGBoost"),
    (rf_bert_probs, "BERT+RF"),
    (hybrid_svm_probs, "Hybrid+SVM"),
    (hybrid_xgb_probs, "Hybrid+XGBoost"),
    (hybrid_rf_probs, "Hybrid+RF"),
    (proposed_probs, "Proposed (Default)"),
    (final_opt_probs, "Proposed (Tuned)"),
]

for probs, label in roc_models:
    fpr, tpr, _ = roc_curve(test_labels_arr, probs)
    roc_auc_val = auc(fpr, tpr)
    plt.plot(fpr, tpr, linewidth=1.8, label=f"{label} (AUC={roc_auc_val:.4f})")

plt.plot([0,1],[0,1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Figure 29: ROC Curve Comparison — All Models")
plt.legend(loc="lower right", fontsize=8)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("Figure29_Master_ROC_Comparison.png", dpi=300)
plt.show()
```

Figure 29: ROC Curve Comparison — All Models




```
# ============================================================
# SECTION 78: FIGURE - PRECISION/RECALL/F1 GROUPED COMPARISON
# ============================================================
x = np.arange(len(master_comparison_df))
width = 0.25

plt.figure(figsize=(15,6))
plt.bar(x - width, master_comparison_df["Precision"], width, label="Precision")
plt.bar(x, master_comparison_df["Recall"], width, label="Recall")
plt.bar(x + width, master_comparison_df["F1"], width, label="F1 Score")

plt.xticks(x, master_comparison_df["Model"], rotation=30, ha="right")
plt.ylabel("Score")
plt.title("Figure 30: Precision, Recall, and F1-Score Comparison — All Models")
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.ylim(master_comparison_df[["Precision","Recall","F1"]].min().min() - 0.003,
         master_comparison_df[["Precision","Recall","F1"]].max().max() + 0.003)
plt.tight_layout()
plt.savefig("Figure30_Master_PRF_Comparison.png", dpi=300)
plt.show()
```

Figure 30: Precision, Recall, and F1-Score Comparison — All Models




```
# ============================================================
# SECTION 79: LIST ALL SAVED ARTIFACTS
# ============================================================
import os

print("All saved files in this session:")
for f in sorted(os.listdir(".")):
    if os.path.isfile(f):
        print(" -", f)
```

```
 - Figure0c_Balanced_Distribution.png
 - Figure12_BERT_Confusion_Matrix.png
 - Figure13_BERT_ROC_Curve.png
 - Figure14_BERT_PR_Curve.png
 - Figure15_BERT_Learning_Curve.png
 - Figure16_BERT_Classifiers_Confusion_Matrices.png
 - Figure17_BERT_Classifiers_ROC_Comparison.png
 - Figure18_Hybrid_Classifiers_Confusion_Matrices.png
 - Figure19_Hybrid_Classifiers_ROC_Comparison.png
 - Figure20_Proposed_Confusion_Matrix.png
 - Figure21_Proposed_ROC_Curve.png
 - Figure22_Proposed_PR_Curve.png
 - Figure23_XGBoost_Estimators_Tuning.png
 - Figure24_XGBoost_MaxDepth_Tuning.png
 - Figure25_RF_Estimators_Tuning.png
 - Figure26_Final_Optimized_Confusion_Matrix.png
 - Figure27_Master_Accuracy_Comparison.png
 - Figure28_Master_F1_Comparison.png
 - Figure29_Master_ROC_Comparison.png
 - Figure30_Master_PRF_Comparison.png
 - Master_Model_Comparison.csv
 - Test_10K.csv
 - Train_40K.csv
 - Tuning_RF_Estimators.csv
 - Tuning_XGBoost_Estimators.csv
 - Tuning_XGBoost_MaxDepth.csv
 - X_test_bert_binary.npy
 - X_test_final_optimized.npy
 - X_test_final_proposed.npy
 - X_test_hybrid.npy
 - X_test_url_features.npy
 - X_train_bert_binary.npy
 - X_train_final_optimized.npy
 - X_train_final_proposed.npy
 - X_train_hybrid.npy
 - X_train_url_features.npy
 - bert_rf_predictions.npy
 - bert_rf_probs.npy
 - bert_svm_predictions.npy
```

```
- hybrid_svm_predictions.npy
- hybrid_svm_probs.npy
- hybrid_xgb_predictions.npy
- hybrid_xgb_probs.npy
- malicious_phish.csv
- proposed_predictions.npy
- proposed_probs.npy
- rf_final_optimized_model.joblib
- test_labels_binary.npy
- train_labels_binary.npy
- xgb_final_optimized_model.joblib
```

```
# ============================================================
# SECTION 80: BUILD ERROR ANALYSIS DATAFRAME
# ============================================================
error_df = pd.DataFrame()
error_df["url"] = test_texts.values
error_df["actual"] = test_labels_arr
error_df["proposed_tuned_pred"] = final_opt_preds
error_df["proposed_tuned_prob_unsafe"] = final_opt_probs

label_map = {0: "Safe", 1: "Unsafe"}
error_df["actual_label"] = error_df["actual"].map(label_map)
error_df["predicted_label"] = error_df["proposed_tuned_pred"].map(label_map)
error_df["is_correct"] = error_df["actual"] == error_df["proposed_tuned_pred"]

print("Total test samples:", len(error_df))
print("Correct predictions:", error_df["is_correct"].sum())
print("Incorrect predictions:", (~error_df["is_correct"]).sum())
```

```
Total test samples: 10000
Correct predictions: 9801
Incorrect predictions: 199
```

```
# ============================================================
# SECTION 81: FALSE POSITIVE / FALSE NEGATIVE BREAKDOWN
# ============================================================
false_positives = error_df[
    (error_df["actual"] == 0) & (error_df["proposed_tuned_pred"] == 1)
].copy()

false_negatives = error_df[
    (error_df["actual"] == 1) & (error_df["proposed_tuned_pred"] == 0)
].copy()

print("False Positives (Safe predicted as Unsafe):", len(false_positives))
print("False Negatives (Unsafe predicted as Safe):", len(false_negatives))

false_positives.to_csv("False_Positives.csv", index=False)
false_negatives.to_csv("False_Negatives.csv", index=False)

print("\nSample False Positives:")
print(false_positives[["url", "proposed_tuned_prob_unsafe"]].head(10))

print("\nSample False Negatives:")
print(false_negatives[["url", "proposed_tuned_prob_unsafe"]].head(10))
```

```
False Positives (Safe predicted as Unsafe): 97
False Negatives (Unsafe predicted as Safe): 102
```